

Chapter 2: Problem Solving and Search Algorithms

By Er. Ukesh Thapa

Syllabus

- 2.1. Formal problem definition: Statesions, transitions, well defined problems
- 2.2. Constraint Satisfaction Problems: Node Consistency, Path Consistency, Backtracking
- 2.3. Search algorithms, Strategies, and Evaluations
- 2.4. Uninformed: BFS, DFS, Iterative Deepening
- 2.5. Informed Search: Best First search, Greedy Search, A* algorithm
- 2.6. Adversarial search: Minimax algorithm, Alpha-beta pruning
- 2.7. Local Search and Optimization: Hill climbing, Simulated Annealing
- 2.8. Evolutionary Optimization: Genetic Algorithm

Hours: 9 Hours

Marks: 12 → (Question type: Question Answer, Numerical)

A point when Problem don't even exist

Before a problem is formed, there is simply The Agent existing in the Environment.

How a "Problem" is born from nothingness?

1. The "Before" Stage: The Reality (The Environment)
2. The Spark: The "Gap" (The Birth of a Need)
3. The Goal Formulation (Naming the Destination)
4. The Problem Formulation (Drawing the Map)

A Big Question: Can we have Universal Problem-Solving Framework?

Background- Philosophy of Search

- ❖ Our goal is to solve every problem in the world
- ❖ Question: Is there an **AI algorithm** that you can give your problem?
- ❖ Our brain has some algorithm that can deal with all kinds of problem.
- ❖ But How does our brain **process** or **create** that **thing** or **algorithm** that it can solve problem of any thing?
 - That is what **AI researcher** are motivated by
- ❖ It try to talk about the **universal algorithm** that can solve any problem
- ❖ That is where the notion of **search** (which is the algorithm) started.
- ❖ If there is one algorithm that can solve every problem, then that means every problem in the world can be converted to this one same input because this is just one algorithm and one algorithm just takes one same input.

If this is true then technically speaking we should have a way to convert all the problems in the world in one input representation.

Background- Philosophy of Search

What is that one uniform representation?

Then I can convert any problem into this one uniform representation and give that as input to one single AI algorithm that this one uniform representation as input and that AI algorithm can solve it.

This question led to the Fundamental concept of **state**.

What is state?

- ❖ All information about the environment.
- ❖ All information necessary to make decision for a task at hand.

Problem-Solving

Problem :

A Problem is simply the **gap** between your **Current State** and your **Goal State** when the path to cross that gap is not obvious.

Problem-Solving:

- ❖ Problem solving in **Artificial Intelligence (AI)** refers to the process by which an **AI system identifies, formulates, and solves problems.**
- ❖ AI problem-solving techniques typically involve defining a problem in terms of states and actions, and then searching through a space of possible solutions (called the state space) to find the best solution.
- ❖ This approach is **foundational in artificial intelligence (AI)** and helps **structure how we think about and solve complex problems.**

Problem-Solving Agents

- ❖ An agent **needs to plan ahead**: to consider a sequence of actions that **form a path to a goal state**.
- ❖ Such an agent is called a **problem-solving agent**, and the **computational process** it undertakes is called **search**.
- ❖ **Romania problem**
 - Imagine an **agent** enjoying a touring vacation in **Romania**.
 - The agent wants to take in the sights, improve its Romanian, enjoy the nightlife, avoid hangovers, and so on.
 - The decision problem is a complex one.
 - Currently agent is in the city of Arad and has a nonrefundable ticket to fly out of Bucharest the following day.

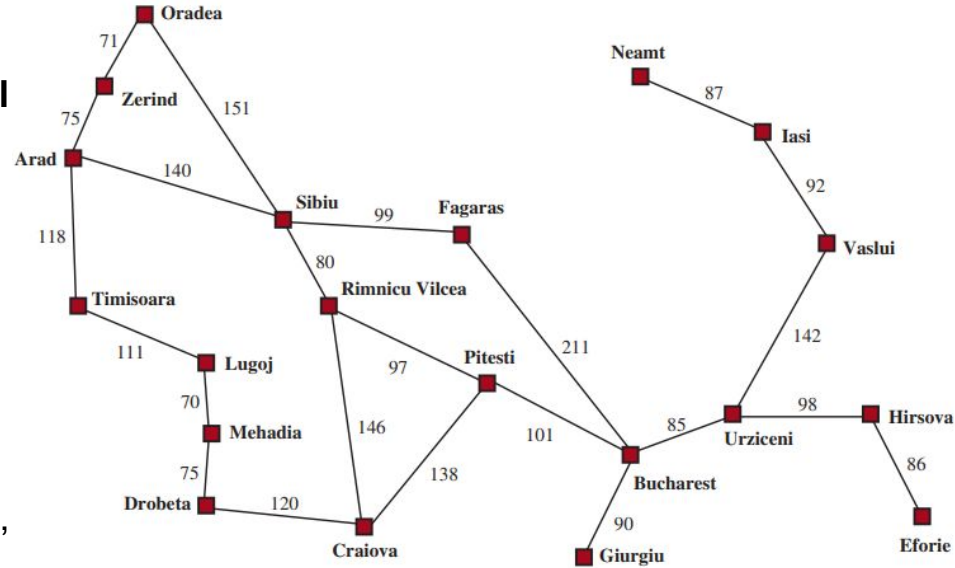


Fig: A simplified road map of part of Romania, with road distances in miles.

Problem-Solving Agents

❖ Romania problem

- The agent observes street signs and sees that there are three roads leading out of Arad:
 - one toward Sibiu,
 - one to Timișoara, and
 - one to Zerind.
- None of these are the goal, so unless the agent is familiar with the geography of Romania, it will not know which road to follow.

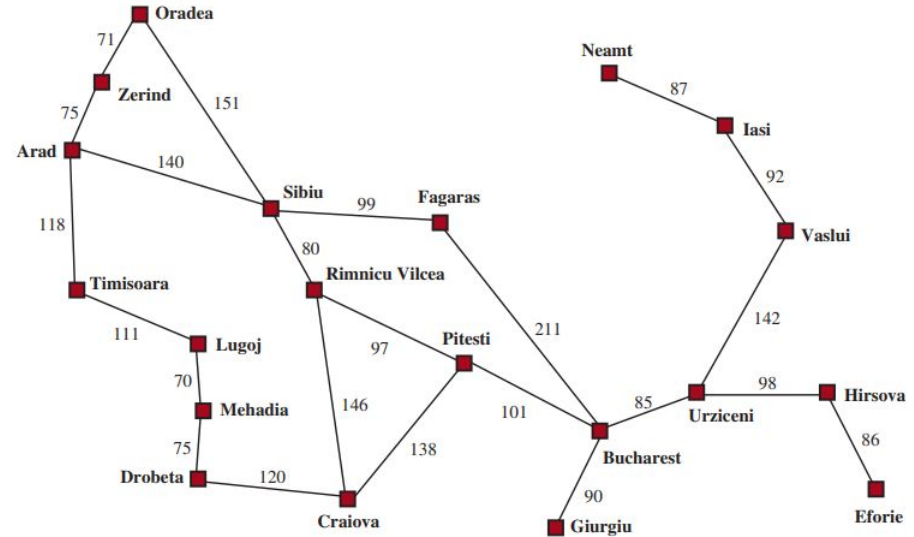


Fig A simplified road map of part of Romania, with road distances in miles.

Problem-Solving Process

There are four-phase problem solving process

- ❖ Goal formulation
 - The agent adopts a "**tunnel vision**" approach. It ignores other factors (like weather, happiness, or other tasks) to focus entirely on reaching one specific state (e.g., "Be in Bucharest").
 - It establishes the set of world states that are considered "success," allowing the agent to know exactly when to stop.
- ❖ Problem formulation
 - The agent strips away real-world details (like the color of the car or scenery) and keeps only what matters (cities and roads).
 - The agent formally defines the "rules of the game": Initial State, Actions, Transition Model, Goal Test, and Path Cost.

Problem-Solving Process

There are four-phase problem solving process

❖ Search

- The agent explores the graph in its "mind" (memory) without moving a single muscle in the real world. It tests paths to see if they work.
- The result of this phase is not just one move, but a complete solution sequence

❖ Execution

- The agent simply follows the solution sequence found in the Search phase, one step at a time
- In simple problem solving, the agent assumes the world hasn't changed while it was thinking

State Space Representation

- ❖ A **set of possible states** that the environment can be in. We call this the **state space**
- ❖ **Initial state:** The starting points or starting conditions
- ❖ **Goal state:** A specific state (or a set of states) in the State Space that satisfies the agent's objective.
- ❖ **Actions:** The processes by which a system changes states.
- ❖ **Transition Model:** Explains what happens when states are subjected to actions.
- ❖ **Path cost:** The expense of moving from an initial state to a certain state, expressed as a numerical value linked to each path.

Problem Representation

- ❖ It is the process of translating your **5 problem components** (Initial State, Actions, Transition, Goal, Cost) into a **visual** and **mathematical structure**.
- ❖ There are two ways we visualize this structure: the **State Space Graph** and the **Search Tree**.
- ❖ **State Space Graph**: This is the **layout** of the **problem**. It **consist** of **nodes** (valid state), **Edges** (represents action), and **loops or cycle**
- ❖ **Search Tree**: This represents the agent's thought process. It is a **data structure** built in the **memory of the AI** as it **explores** the **map**. It consist of **Root node** (starts with initial state), **Branches** (it add new branch every time agent consider action), and **Duplicates** (state can be appear multiple time)

State Space Graph vs Search Tree

Feature	State Space Graph	Search Tree
What is it?	The Territory (The physical world).	The Exploration (The path taken).
Status	Static . It is fixed and defined by the problem .	Dynamic . It grows as the algorithm runs .
Uniqueness	Each state appears once .	A state can appear multiple times (duplicates).
Loops	Loops are allowed. (You can go $A \rightarrow B \rightarrow A$).	No loops . A tree structure only flows down. A "loop" in the graph becomes an infinite branch in the tree.
Size	Finite (usually).	Can be Infinite (if the algorithm doesn't stop loops).

Well-Defined Problems and Solutions

- ❖ Well-defined problems have **specific end goals** and **clearly expected solutions**, while **ill-defined problems do not**.
- ❖ Well-defined problems allow for more initial planning than ill-defined problems
- ❖ Well-defined problems should follow **5 component checklist**
 - **Initial state, Actions, Transition model, Goal Test, and Path cost**
- ❖ A solution is NOT the Goal State itself.
- ❖ A solution is the Sequence of Actions that leads from the Start to the Goal. (The solution is "Drive to Sibiu, then Drive to Fagaras, then Drive to Bucharest").
- ❖ Example Problem:
 - Toy problem: The vacuum world, 8 puzzle, and 8 queen problem
 - Real world problem: Route Finding (GPS), The Traveling Salesperson Problem (TSP)

Constraint Satisfaction Problems (CSP)

- ❖ A **CSP** is a **mathematical problem** defined **not by the algorithm** used to solve it, but by the **structure of the data**.
- ❖ Instead of saying "**how do I get there?**" (like pathfinding), a CSP asks "**what state satisfies all the rules?**"
- ❖ A CSP is formally defined by three components (X, D, C):
 - Variables (X): A set of variables $\{X_1, X_2, \dots, X_n\}$.
 - Example: In $TWO + TWO = FOUR$, the variables are the letters T, W, O, F, U, R.
 - Domains (D): The set of allowed values for each variable.
 - Example: Each letter represents a digit from $\{0, 1, \dots, 9\}$.
 - Constraints (C): Rules that limit the values variables can take.

Constraint Propagation

- ❖ In **standard backtracking**, we make a **choice** and then **check if it's wrong**.
- ❖ **Constraint Propagation** is a **smarter** approach: it **looks ahead** and **removes** (prunes) values from **domains** that cannot possibly be **part of a solution**, **reducing** the **search space** before we **even start guessing**.

Why consistency?

- **Without Consistency:** You try 0000, 0001, 0002... **blindly until one works**. This is **slow**.
- **With Consistency:** You look at the **rules first**. If Rule #1 says "The first digit is odd," you immediately throw away 0xxx, 2xxx, 4xxx... **without ever testing them**.

Consistency is important because it "prunes" (cuts off) massive sections of the search tree before you waste time exploring them. It transforms a problem from "impossible to solve in a million years" to "solvable in seconds."

Types of Consistency

❖ Node Consistency

- This is the **simplest form**. A **single variable** is node-consistent if all the values in its domain satisfy its **unary constraints** (constraints that involve only that variable).
- Definition: A variable X is node-consistent if for every value x in its domain D_X , the unary constraints on X are satisfied.
- **Example:**
 - You are **assigning** a task to a **Worker**. The domain is {Monday, Tuesday, Wednesday}
 - **Constraint:** The worker is sick on Monday.
 - **Without Consistency:** Your algorithm tries to schedule Monday, fails, backtracks, and tries again.
 - **With Consistency:** You remove Monday instantly. The domain is now {Tue, Wed}. You never waste time thinking about Monday.

Types of Consistency

❖ Arc Consistency (2-Consistency)

- This is the most popular consistency technique. It deals with **binary constraints** (constraints between **two variables**).
- Definition: A variable **X is arc-consistent** with respect to another **variable Y** if, for every value x in the domain of X , there exists at least one value y in the domain of Y that satisfies the binary constraint between them.
- Example:
 - Suppose we have variables A and B with domains $\{1, 2, 3\}$ and constraint $A < B$.
 - If $A = 3$, can B be anything? No, because B cannot be greater than 3 (max domain is 3).
 - Action: Remove 3 from $\text{Domain}(A)$.
 - Now A is arc-consistent with respect to B .

Types of Consistency

❖ Path Consistency

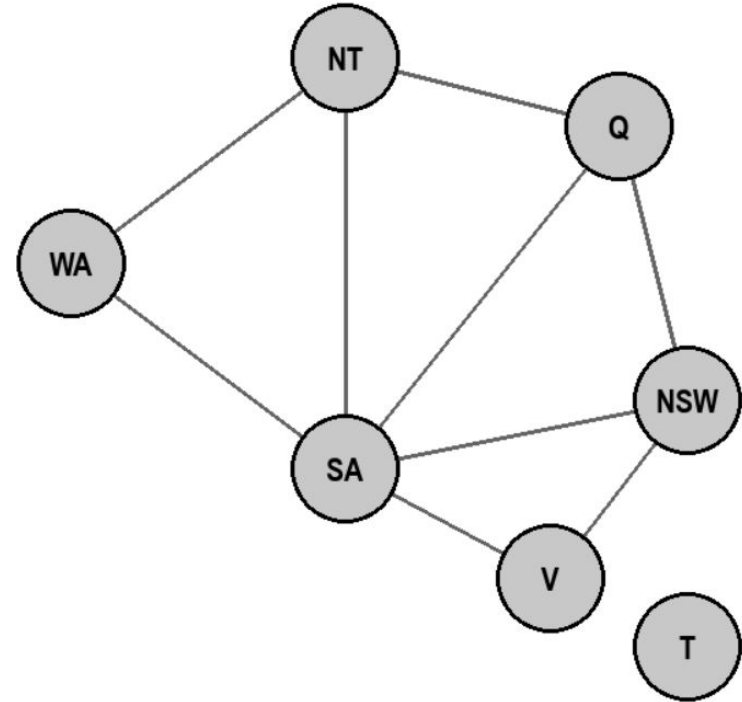
- This looks at **triples of variables**. It ensures that **binary constraints** are consistent with respect to a **third variable**.
- Definition: A pair of variables $\{X, Y\}$ is path-consistent with respect to a third variable Z if, for every consistent assignment $\{X=x, Y=y\}$, there is a value z in D_Z that satisfies the constraints between $\{X, Z\}$ and $\{Y, Z\}$.
- Why use it? Arc consistency isn't enough to catch all impossibilities. Path consistency tightens the binary constraints by inferring new constraints from a third variable.
- This is **computationally expensive** and used **less frequently** than **Arc Consistency**, but it infers implicit constraints that simpler methods miss.

Backtracking Algorithm

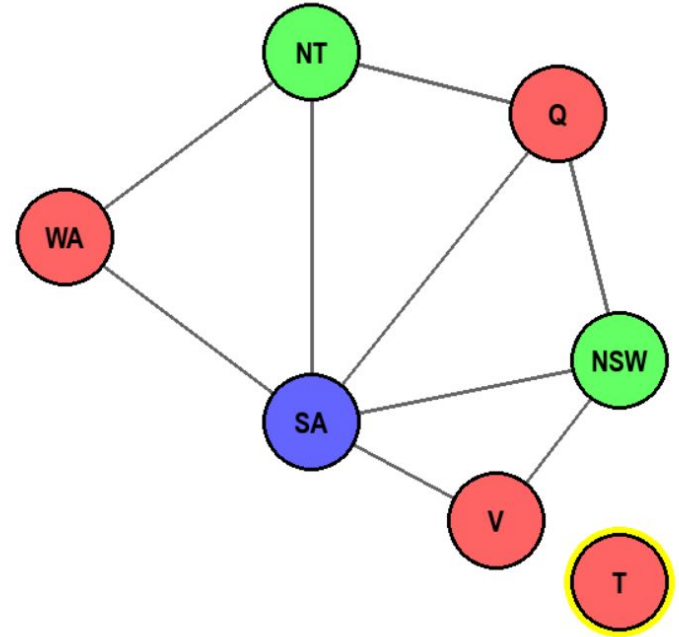
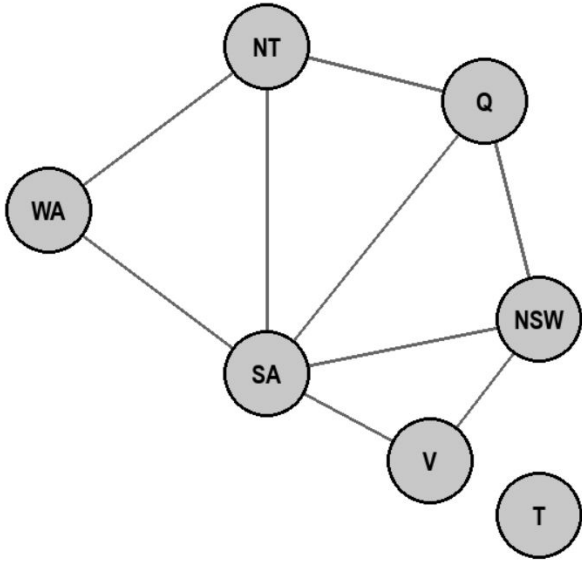
- ❖ Backtracking is a **problem-solving algorithmic** technique that involves **finding a solution** incrementally by trying **different options** and **undoing them if they lead to a dead end**.
- ❖ It is commonly used in **situations** where you need to explore **multiple possibilities** to **solve a problem**
- ❖ **Example:** searching for a path in a maze or solving puzzles like Sudoku.
- ❖ When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted.
- ❖ Example: Finding the **shortest path through a maze**
 - **Algorithm:**
 - **Step 1: Start** at the starting point.
 - **Step 2:** For each of the four possible directions (up, down, left, right), try moving in that direction.
 - **Step 3:** If moving in that **direction leads** to the ending point, **return the path taken**.
 - **Step 4:** If moving in that direction does not lead to the ending point, backtrack to the previous position and try a different direction.
 - **Step 5:** Repeat steps 2-4 until the ending point is reached or all possible paths have been explored.

CSP: Map Coloring

- ❖ The Rules:
 - **Variables:** The regions on the map (e.g., Region A, Region B, Region C).
 - **Domain:** The colors you are allowed to use (e.g., {Red, Green, Blue}).
 - **Constraint:** If two regions touch each other (share a border), they cannot be the same color.



CSP: Map Coloring



CSP: N-Queens Problem

- ❖ Variables: The positions of the queens on the chessboard (usually an $n \times n$ board).
- ❖ Domains: Each queen can be placed in one of the rows or columns of the chessboard.
- ❖ Constraints:
 - Only one queen per row.
 - Only one queen per column.
 - No two queens can share a diagonal.

CSP: N-Queens Problem

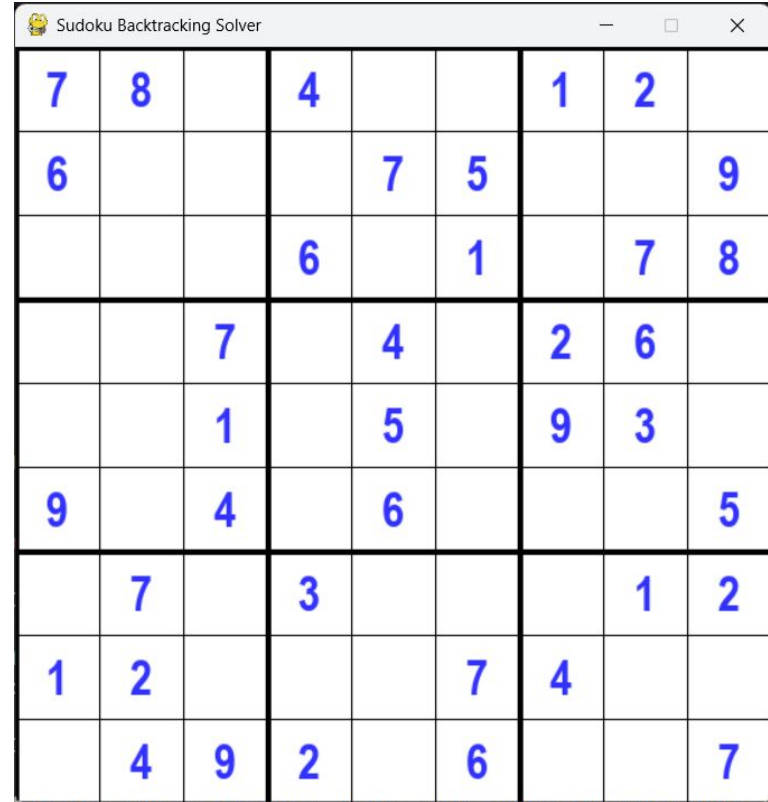
❖ The Backtracking Logic

We solve this column by column (from left to right, Column 0 to 7).

1. Start at Column 0.
2. Try Row 0: Place a Queen.
3. Check Safety: Is she under attack by any queen to her left? (Since we haven't placed any to the right yet, we only look left).
 - a. If Safe: Move to Column 1.
 - b. If Unsafe: Remove her and try Row 1.
4. Backtrack: If we try all 8 rows in a column and none are safe, it means our placement in the previous column was wrong. We go back, move the previous queen, and try again.

CSP: CSP: Sudoku Problem

- ❖ The Rules:
 - Row Rule: Every row must contain numbers 1 through 9 exactly once.
 - Column Rule: Every column must contain numbers 1 through 9 exactly once.
 - Box Rule: Every 3x3 sub-grid must contain numbers 1 through 9 exactly once.



A screenshot of a window titled "Sudoku Backtracking Solver". The window displays a 9x9 Sudoku grid. The numbers in the grid are as follows:

7	8		4			1	2	
6				7	5			9
			6		1		7	8
		7		4		2	6	
		1		5		9	3	
9		4		6				5
	7		3				1	2
1	2				7	4		
	4	9	2		6			7

CSP: Sudoku Problem

- ❖ Algo:
 - Find Empty Cell: Look for a cell containing 0 (or empty).
 - Try Numbers 1-9:
 - Check Row: Is the number already in this row?
 - Check Column: Is the number already in this column?
 - Check Box: Is the number already in the 3 X 3 box?
 - Place & Recurse: If the number is valid, place it and call the solver for the next empty cell.
 - Backtrack: If the recursive call returns False (meaning we got stuck later), reset this cell to 0 and try the next number.

7	8	5	4	3	9	1	2	6
6	1	2	8	7	5	3	4	9
4	9	3	6	2	1	5	7	8
8	5	7	9	4	3	2	6	1
2	6	1	7	5	8	9	3	4
9	3	4	1	6	2	7	8	5
5	7	8	3	9	4	6	1	2
1	2	6	5	8	7	4	9	3
3	4	9	2	1	6	8	5	7

CSP: Cryptarithmic

- **Cryptarithmic** is a **puzzle consisting** of an **arithmetic problem** in which **the digits** have been **replaced** by letters of the **alphabet**.
- The **goal** is to **decipher** the letters (ie. map them back onto the digits) using the constraints provided by **arithmetic** and the **additional constraint** that **no two letters can have the same numerical value**

CSP: Cryptarithmic Problems

$$\text{SEND} + \text{MORE} = \text{MONEY}$$

Variables: S, E, N, D, M, O, R, Y.

Domains: $\{0, 1, 2, \dots, 9\}$ for each variable.

Constraints:

- All letters must be assigned a unique digit (no two letters can share the same digit).
- The equation $\text{SEND} + \text{MORE} = \text{MONEY}$ must hold true.

Examples

Eg. Crypto-arithmetic

- i. $\text{SEND} + \text{MORE} = \text{MONEY}$
- ii. $\text{RIGHT} + \text{RIGHT} = \text{WRONG}$
- iii. $\text{WRONG} + \text{WRONG} = \text{RIGHT}$
- iv. $\text{LOGIC} + \text{LOGIC} = \text{PROLOG}$
- v. $\text{CROSS} + \text{ROADS} = \text{DANGER}$
- vi. $\text{BASE} + \text{BALL} = \text{GAMES}$
- vii. $\text{ONE} + \text{ONE} = \text{TWO}$
- viii. $\text{ONE} + \text{ONE} + \text{TWO} = \text{FOUR}$
- ix. $\text{KYOTO} + \text{OSAKA} = \text{TOKYO}$
- x. $\text{APPLE} + \text{GRAPE} = \text{CHERRY}$

Solving

$$\begin{array}{r} \text{TO} \\ + \text{GO} \\ \hline \text{OUT} \end{array}$$

Carry then easy O=1

Letter	Digit
T	2
O	1
G	
U	

Solving

$$\begin{array}{r} \text{TO} \\ + \text{GO} \\ \hline \text{OUT} \end{array}$$

$$2 + G = U + 10$$

Or

$$2 + 9 = 11 \quad \text{-----} \quad \text{X}$$

Or

$$2 + 8 = 10 \quad \text{-----} \quad \text{correct}$$

Letter	Digit
T	2
O	1
G	8
U	0

Solving

For Carry

M=1

$$\begin{array}{rcccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

$$\begin{array}{rcccc} & \square & \square & \square & \square \\ + & 1 & \square & \square & \square \\ \hline 1 & \square & \square & \square & \square \end{array}$$

Letter	Digit
S	
E	
N	
D	
M	1
O	
R	
Y	

Solving

- Case 1:
 - No carry, i.e., $c3=0$
- Case 2:
 - Carry, i.e., $c3=1$
- Moving with $c3=0$
 - $S + 1 = 10 \rightarrow s=9$

$$\begin{array}{rcccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

$$\begin{array}{rcccc} & c3 & c2 & c1 & \\ & \boxed{9} & \boxed{} & \boxed{} & \boxed{} \\ + & \boxed{1} & \boxed{} & \boxed{} & \boxed{} \\ \hline \boxed{1} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \end{array}$$

Letter	Digit
S	9
E	
N	
D	
M	1
O	
R	
Y	

Solving

- Moving with $c3=0$
 - $S + 1 = 10 \rightarrow s=9$
 - $O = 0$
- Case 3: $c2=0$
- Case 4: $c2=1$

$$\begin{array}{rcccc} & S & E & N & D \\ + & M & O & R & E \\ \hline M & O & N & E & Y \end{array}$$

Take case 3, i.e., $c2=0$

- $E+O=N \rightarrow$ not possible

Take case 4, i.e., $c2=1$

$$\begin{array}{rcccc} & c3 & c2 & c1 & \\ & \boxed{9} & \boxed{} & \boxed{} & \boxed{} \\ + & \boxed{1} & \boxed{0} & \boxed{} & \boxed{} \\ \hline \boxed{1} & \boxed{0} & \boxed{} & \boxed{} & \boxed{} \end{array}$$

Letter	Digit
S	9
E	
N	
D	
M	1
O	0
R	
Y	

Solving

Case 5: Let E= 5 then

$$5 + 0 + 1(c2) = N$$

$$N=6$$

	S	E	N	D	
+	M	O	R	E	

	M	O	N	E	Y

		c3	c2	c1	
		9	5	6	
+		1	0		5

	1	0	6	5	

Letter	Digit
S	9
E	5
N	6
D	
M	1
O	0
R	
Y	

Solving

Case 5: Let $E = 5$ then

$$5 + 0 + 1(c_2) = N$$

$$N = 6$$

Case 6: $c_1 = 0$

Case 7: $c_1 = 1$

Moving with case 6, i.e., $c_1 = 0$

$$6 + R = 5 \text{ and } c_2 = 1 \text{ } R = 9 \text{ not possible}$$

Moving with case 7, i.e., $c_1 = 1$

Take $R = 8$ is possible

$$\begin{array}{r} \text{S} \quad \text{E} \quad \text{N} \quad \text{D} \\ + \quad \text{M} \quad \text{O} \quad \text{R} \quad \text{E} \\ \hline \text{M} \quad \text{O} \quad \text{N} \quad \text{E} \quad \text{Y} \end{array}$$

$$\begin{array}{r} \quad c_3 \quad c_2 \quad c_1 \\ \quad \boxed{9} \quad \boxed{5} \quad \boxed{6} \quad \boxed{} \\ + \quad \quad \boxed{1} \quad \boxed{0} \quad \boxed{8} \quad \boxed{5} \\ \hline \boxed{1} \quad \boxed{0} \quad \boxed{6} \quad \boxed{5} \quad \boxed{} \end{array}$$

Letter	Digit
S	9
E	5
N	6
D	
M	1
O	0
R	8
Y	

Solving

Case 5: Let E = 5 then

$$5 + 0 + 1(c_2) = N$$

$$N = 6$$

Case 6: $c_1 = 0$

Case 7: $c_1 = 1$

Moving with case 6, i.e., $c_1 = 0$

$$6 + R = 5 \text{ and } c_2 = 1 \text{ } R = 9 \text{ not possible}$$

Moving with case 7, i.e., $c_1 = 1$

Take $R = 8$ is possible

$$5 + D = 10 + Y$$

$D = 6, 5, 4, 3, 2$ not possible

Take $D = 7$ then $Y = 2$ which is possible

$$\begin{array}{r} \text{S} \quad \text{E} \quad \text{N} \quad \text{D} \\ + \quad \text{M} \quad \text{O} \quad \text{R} \quad \text{E} \\ \hline \text{M} \quad \text{O} \quad \text{N} \quad \text{E} \quad \text{Y} \end{array}$$

$$\begin{array}{r} \quad \quad c_3 \quad c_2 \quad c_1 \quad \\ \quad \quad \boxed{9} \quad \boxed{5} \quad \boxed{6} \quad \boxed{7} \\ + \quad \quad \boxed{1} \quad \boxed{0} \quad \boxed{8} \quad \boxed{5} \\ \hline \boxed{1} \quad \boxed{0} \quad \boxed{6} \quad \boxed{5} \quad \boxed{2} \end{array}$$

Letter	Digit
S	9
E	5
N	6
D	7
M	1
O	0
R	8
Y	2

Search Algorithms

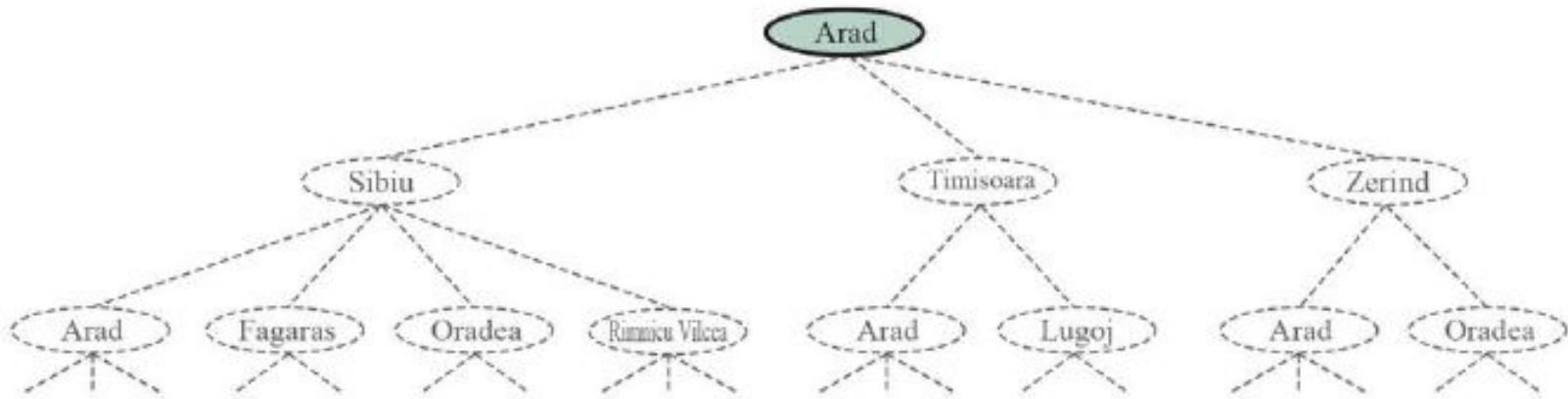
- ❖ A **search algorithm** takes a **search problem** as input and returns a solution, or an indication of failure.
- ❖ In this chapter we consider algorithms that superimpose a **search tree** over the **state-space graph**, forming various paths from the initial state, trying to find a path that reaches a goal state.
- ❖ Each **node** in the search tree corresponds to a **state** in the **state space** and the **edges** in the **search tree** correspond to **actions**.
- ❖ The **root** of the tree corresponds to the initial state of the problem.

Search Algorithms

- ❖ It is important to understand the **distinction** between the **state space** and the **search tree**.
- ❖ The **state space** describes the (possibly infinite) **set of states in the world**, and **the actions** that allow **transitions** from **one state to another**.
- ❖ The **search tree** describes **paths between these states**, reaching towards **the goal**.
- ❖ **The search tree** may have **multiple paths** to (and thus multiple nodes for) **any given state**, but each **node in the tree** has a **unique path back** to the **root** (as in all trees).

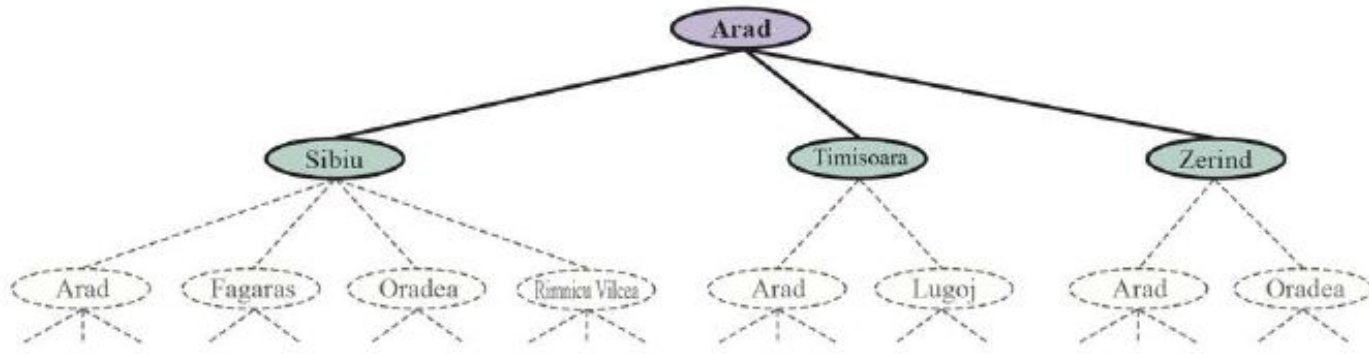
Search Algorithms

- ❖ The first few steps in finding a path from **Arad** to **Bucharest**
- ❖ The **root node** of the search tree is at the **initial state, Arad**.



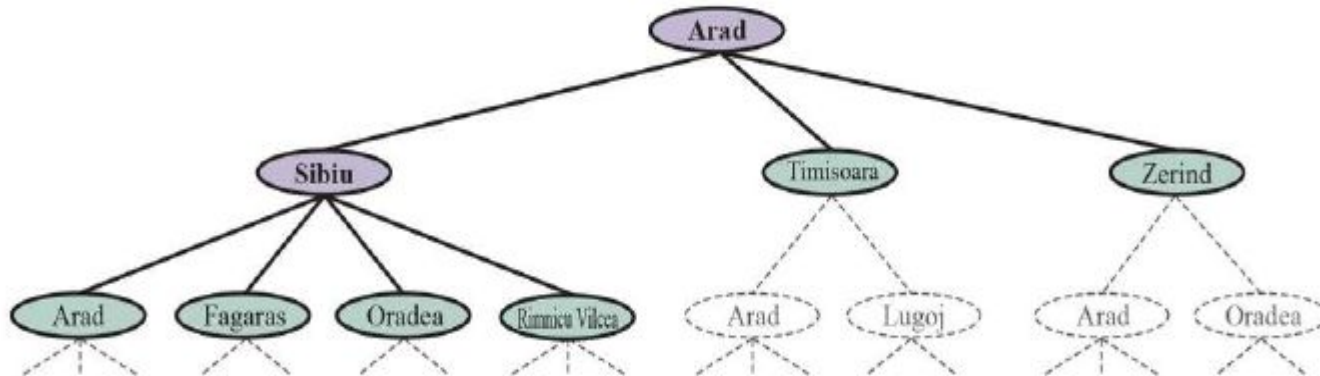
Search Algorithms

- ❖ We can expand the node, by considering the
 - Available **ACTIONS** for that **state**, using the **RESULT** function to see where those **actions lead to**, and
 - Generating a new node (called a child node or successor node) for each of the resulting states.
- ❖ Each child node has **Arad** as its **parent node**.



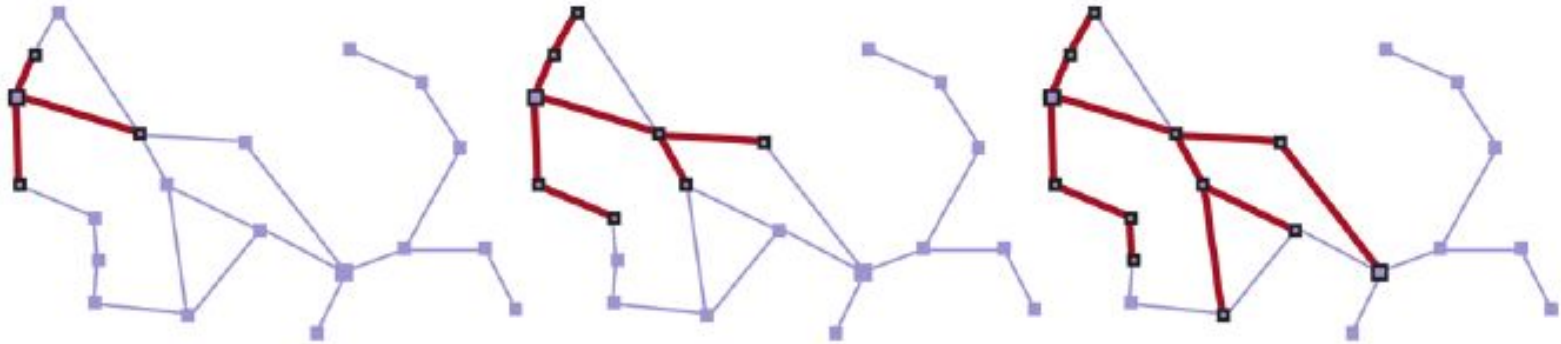
Search Algorithms

- ❖ Now we must choose which of these **three child nodes** to **consider next**. This is the essence of search—following up one option now and putting the others aside for later.
- ❖ Suppose we choose to expand **Sibiu first**
- ❖ Figure (bottom) shows the result: a set of 6 unexpanded nodes (outlined in bold). We call this the frontier of the search tree.



Search Algorithms

- ❖ Figure below shows the search tree superimposed on the state-space graph.



Search Strategy/Algorithms

- ❖ A **search strategy** is defined by **picking** the order of **node expansion**
- ❖ Search strategy are evaluated on the basis of following:
 - **Completeness:** An **algorithm** is said to be **complete** if it definitely **finds solution to the problem**, if exist.
 - **Time Complexity:** How **long** (worst or average case) does it take to find a **solution**? Usually measured in terms of the number of nodes generated or expanded.
 - **Space Complexity:** How much **space** is used by the **algorithm**? Usually measured in terms of the maximum number of nodes in memory at a time
 - **Optimality/Admissibility:** If a solution is found, is it guaranteed to be an optimal one? For example, is it the one with **minimum cost**

Search Strategy/Algorithms

❖ Two Types:

- Uninformed Search Algorithm
- Informed Search Algorithm

Informed vs Uninformed Search

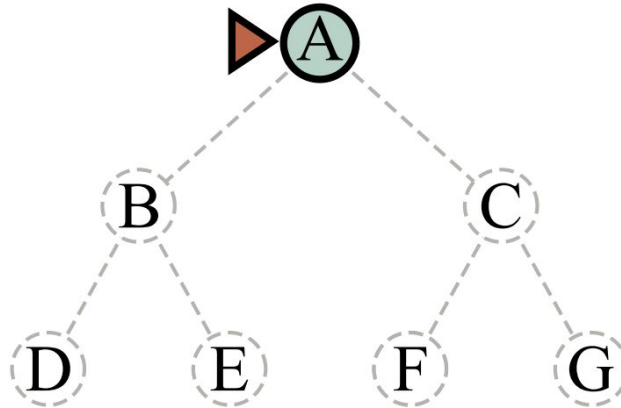
Informed Search	Uninformed Search
It uses knowledge for the searching process.	It doesn't use knowledge for searching process
It finds solution more quickly	It finds solution slow as compared to informed search
It is highly efficient	It is mandatory efficient
Cost is low	Cost is high
It consumes less time	It consumes moderate time
It provides the direction regarding the solution	No suggestion is given regarding the solution it.
Greedy Search, A* Search, Graph Search	Depth First Search, Breadth First Search

Uninformed Search Strategies

- ❖ **Breadth-First Search (BFS)**
- ❖ **Depth-First Search (DFS)**
- ❖ **Iterative Deepening Search**

Breadth-First Search (BFS)

- ❖ **All nodes** are **expanded** at a given **depth** in the search tree before **any nodes** at the **next level** are expanded until **the goal reached**.
- ❖ Expands the **shallowest nodes** first
- ❖ It uses a **FIFO** (First-In, First-Out) Queue.
- ❖ It explores **level-by-level** (Layer 1, then Layer 2, etc.).



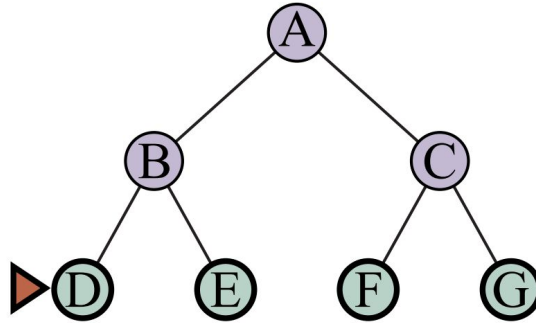
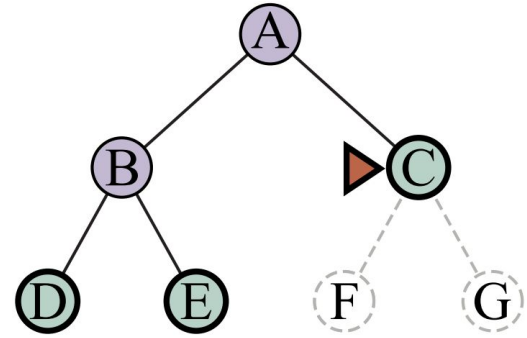
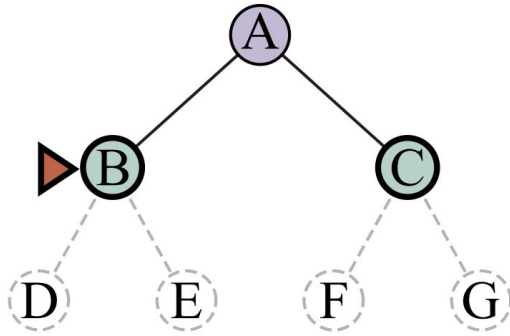
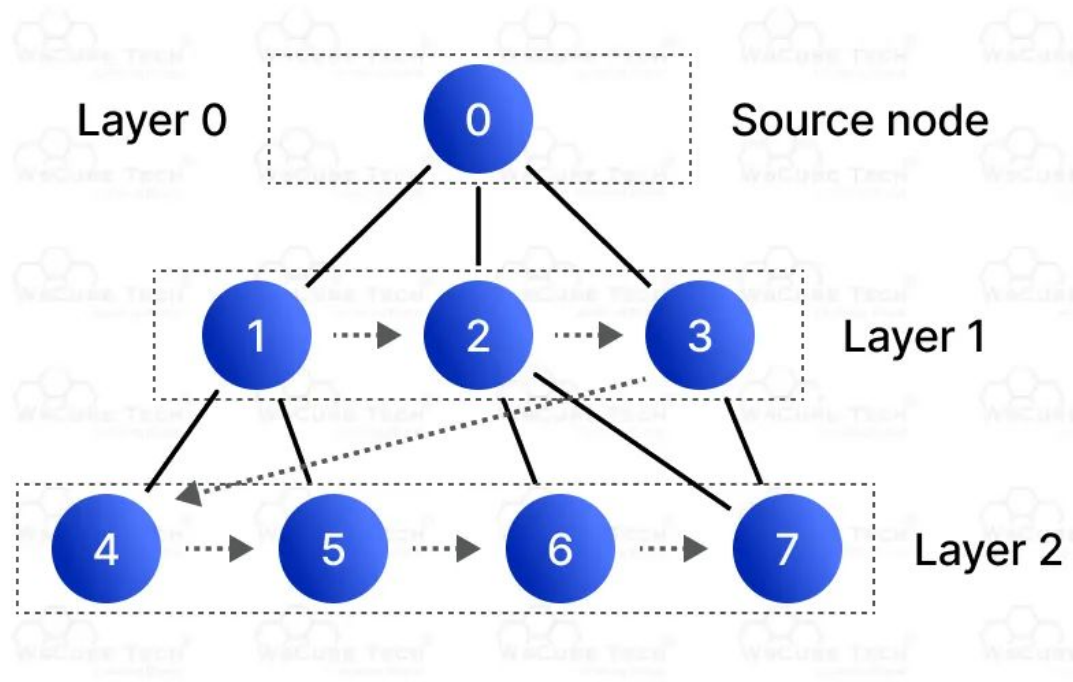


Figure: Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

Breadth-First Search (BFS)



BFS: Evaluation

Completeness:

- ❖ **Does** it always find **a solution** if one exists?
- ❖ **YES**
- ❖ The **Breadth-First Search** (BFS) algorithm is considered **complete**, meaning that it is **guaranteed** to find a **solution** if **one exists**, provided that certain conditions are met.
 - if a solution exists at a certain depth, BFS will find it before moving deeper into the search space.

BFS: Evaluation

Time Complexity?

- ❖ b = Branching factor, d = depth
- ❖ Total Nodes Generated: $1 + b + b^2 + b^3 + \dots + b^d = O(b^d) \rightarrow$ **Big O notation**
- ❖ Exponential **time growth**.

BFS: Evaluation

Space Complexity?

Memory

BFS: Evaluation

Space Complexity?

- ❖ The maximum number of nodes that must be stored in memory (RAM) at any single moment during the search.
- ❖ Total no. of nodes in memory: $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$

BFS requires exponential **space** $O(b^d)$ because it stores the entire level

BFS: Evaluation

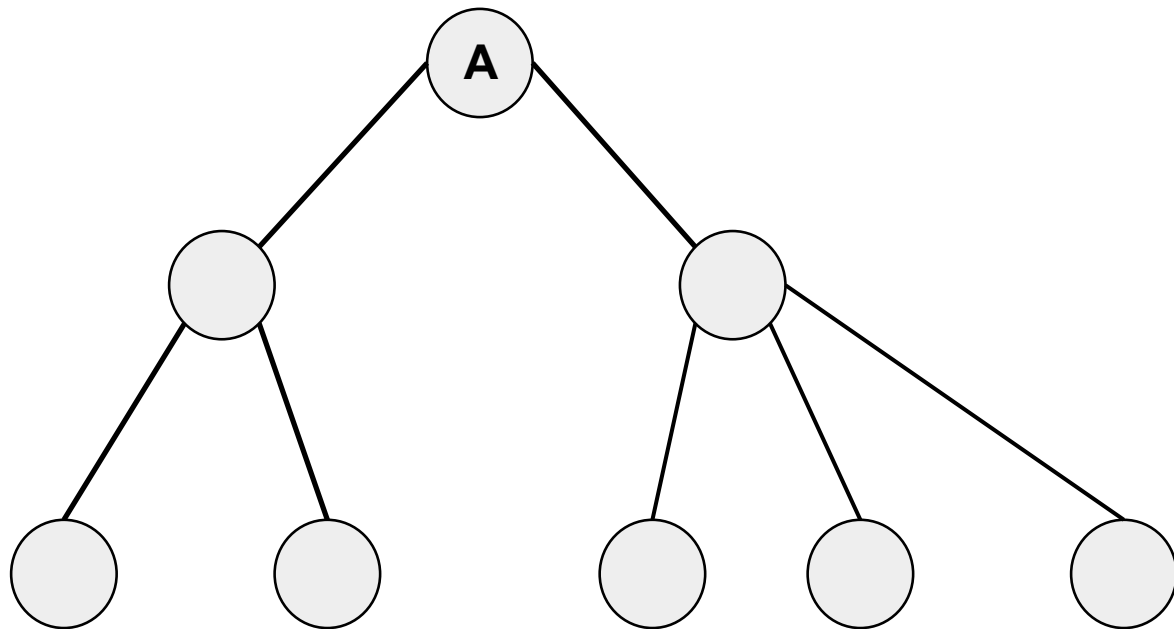
Optimal?

- ❖ If **all paths** have the **same cost**. Otherwise, **not optimal but** finds **solution** with **shortest path length** (shallowest solution).
- ❖ If each path does not have **same path** cost shallowest **solution** may **not be optimal**.

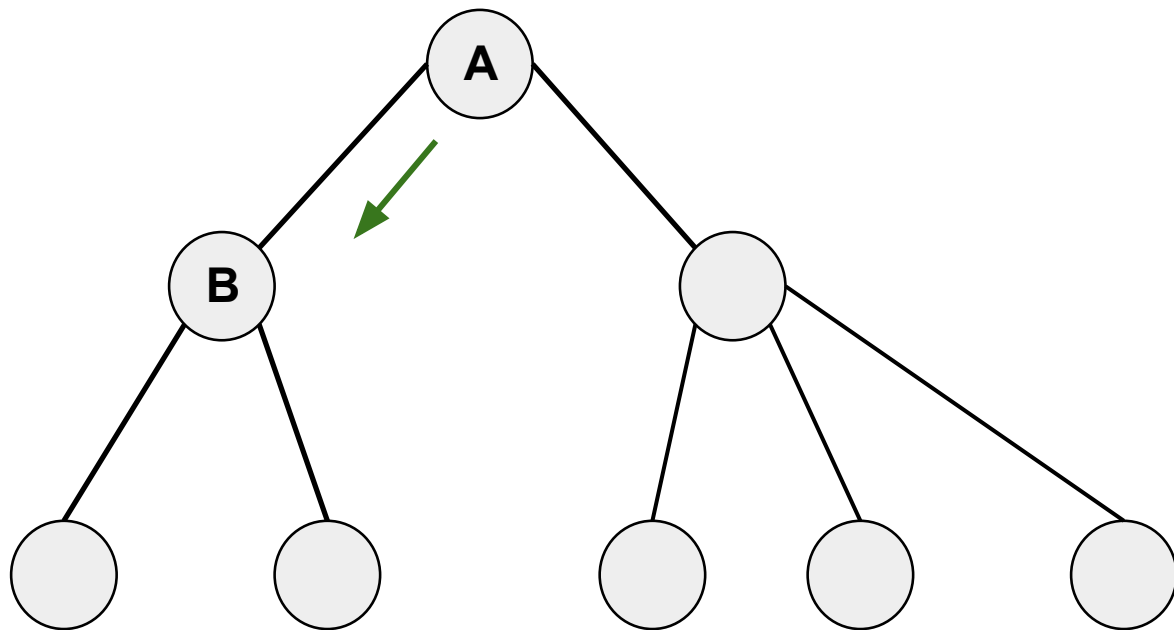
Depth-First Search (DFS)

- ❖ Looks for the **goal node** among **all the children** of the **current node** before using the **sibling of this node** i.e. expand deepest unexpanded node.
- ❖ Implemented as a **LIFO queue** (=stack)

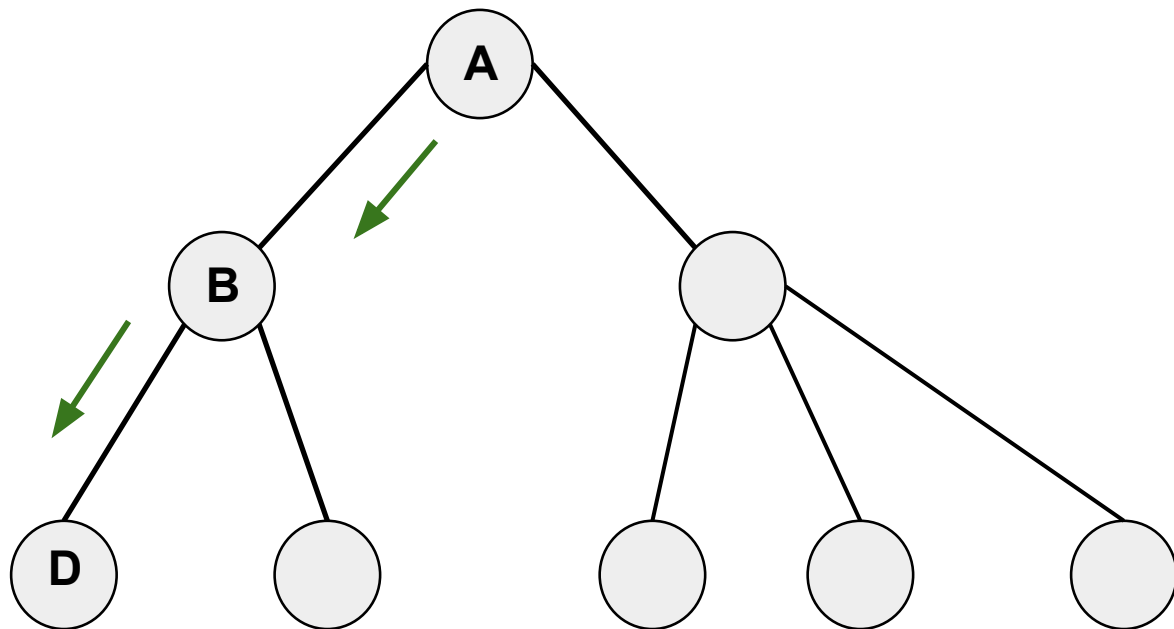
DFS



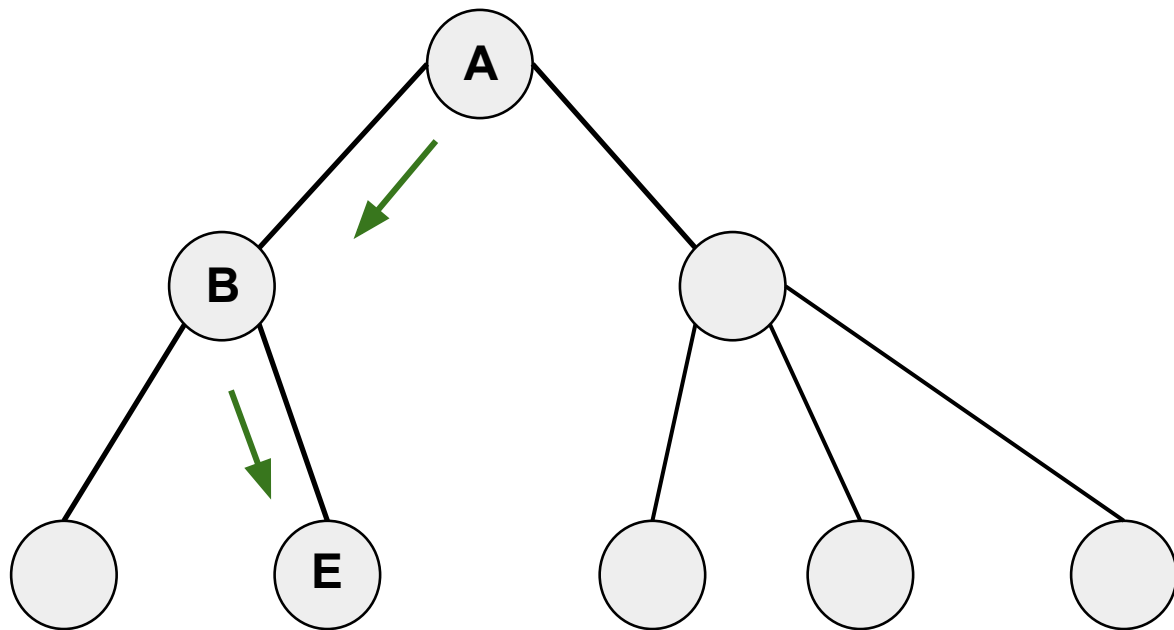
DFS



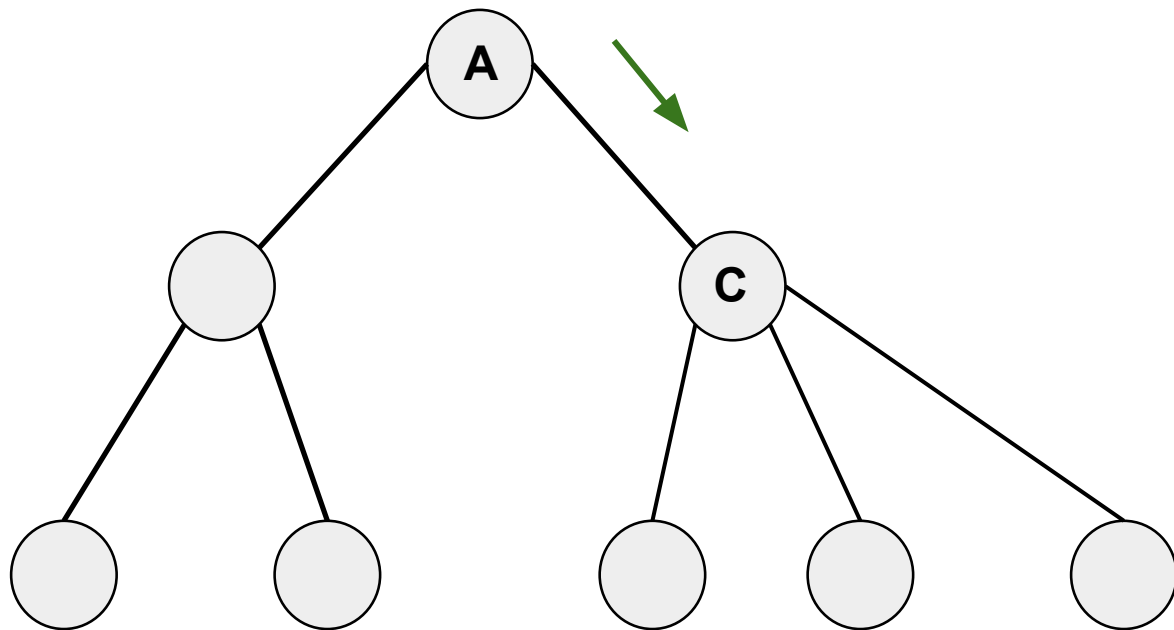
DFS



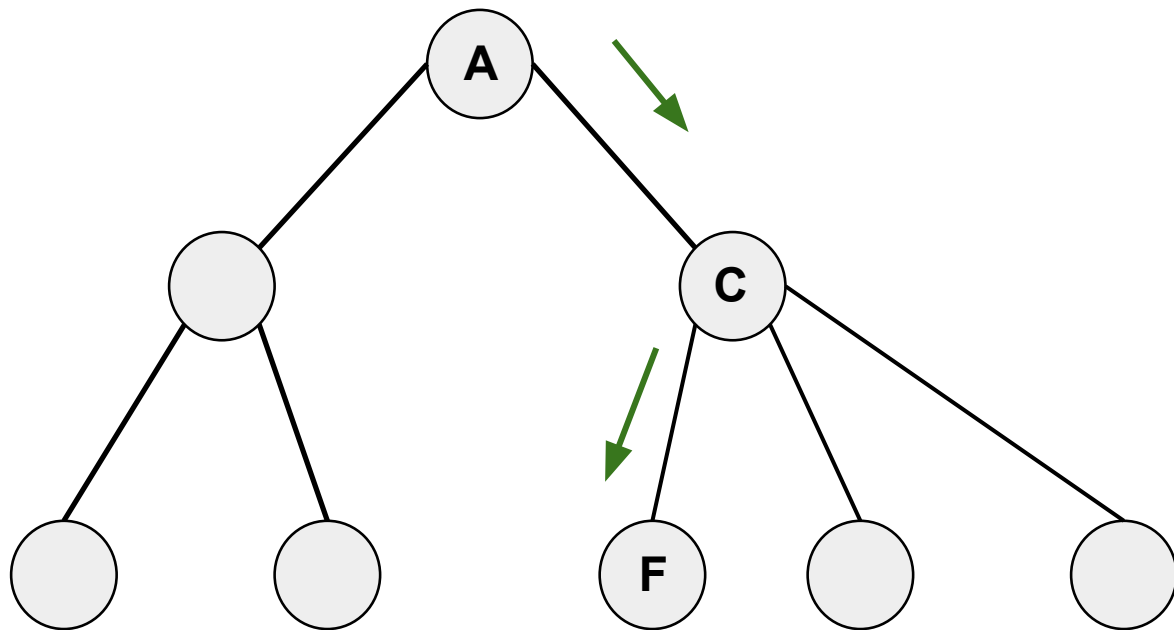
DFS



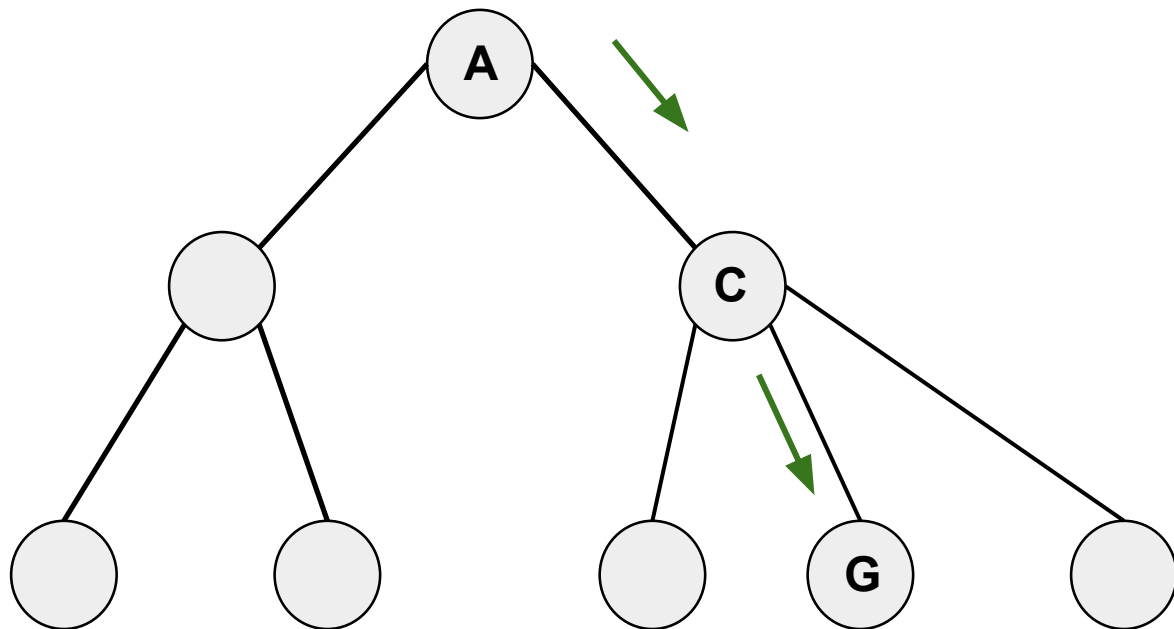
DFS



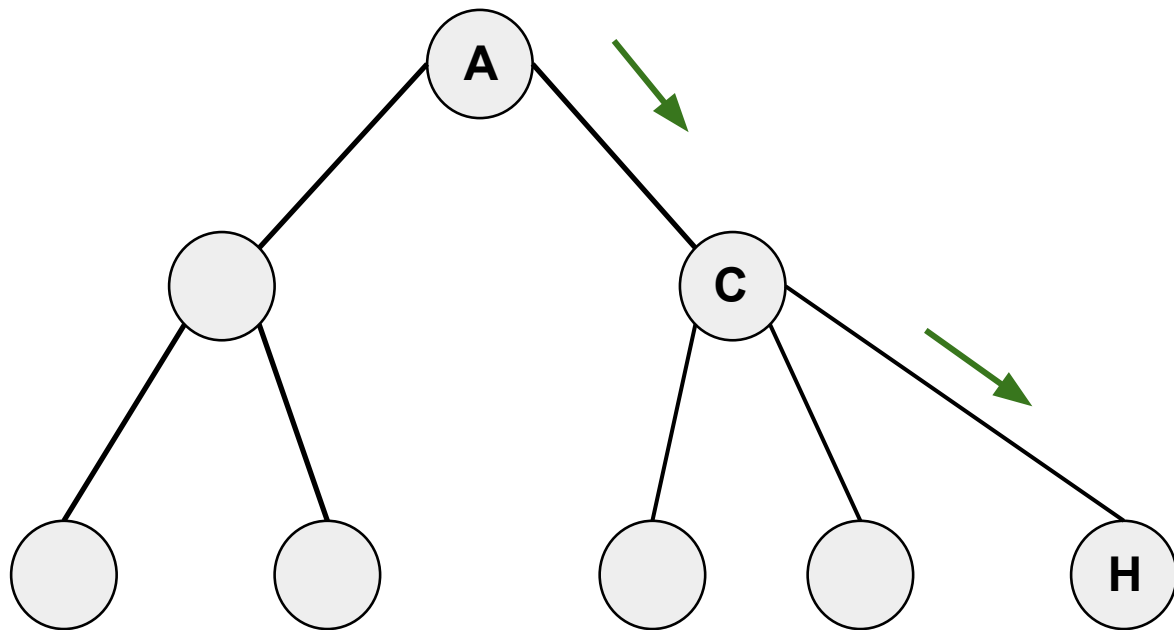
DFS



DFS

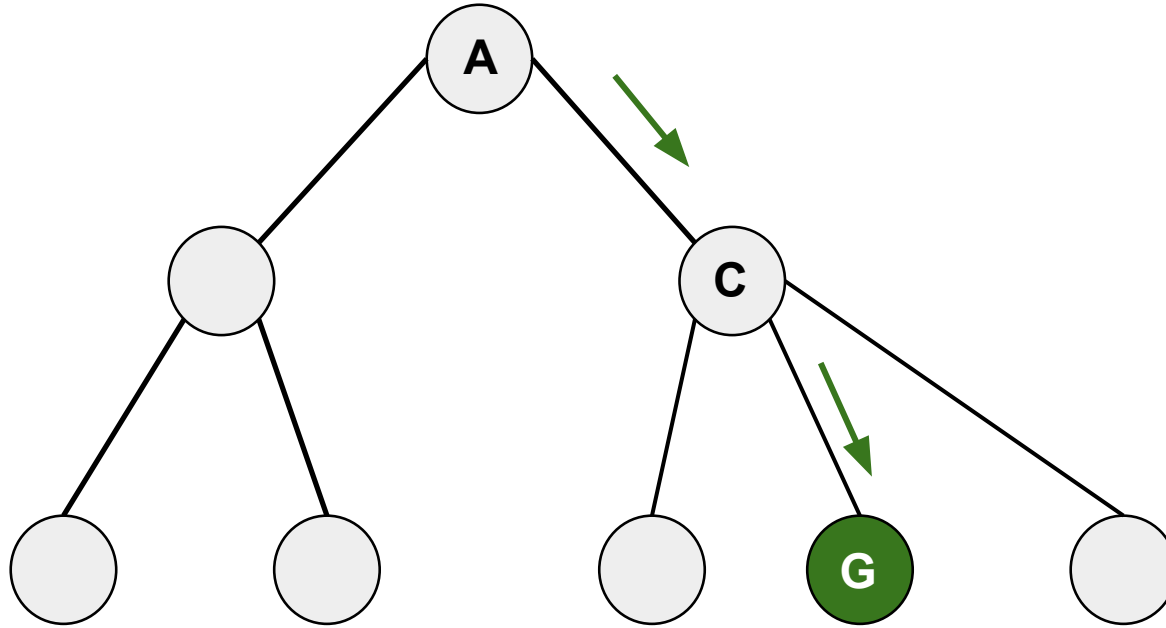


DFS



DFS

If goal is **g**, the path is?



DFS: Evaluation

Time complexity

- ❖ Let m is the maximum depth of the search tree.
 - Depth 0: Number of nodes = 1
 - Depth 1: Number of nodes = b
 - Depth 2: Number of nodes = b^2
- ❖ **Worst case:** My goal **node** is on the **last right node**
- ❖ Root has b successors, each node at the next level has again b successors (total Worst case; expand all except the last node at depth m)
- ❖ **Total no. of nodes generated:** $b + b^2 + b^3 + \dots + b^m = O(b^m)$

DFS: Evaluation

Space complexity

- ❖ It **needs to store** only **a single path** from the **root node** to a **leaf node**, along with **remaining unexpanded sibling nodes** for each node on the path.
- ❖ Total no. of nodes in memory = $1 + b + b + b + \dots + b$ m times $O(bm)$
- ❖ Where b is the branching factor and m is the maximum depth of the tree

DFS: Evaluation

Optimal (i.e., admissible):

- ❖ **DFS** expand deepest **node first**, it expands entire **left sub-tree even** if **right subtree contains goal nodes at levels 2 or 3**.
- ❖ Thus we can say **DFS may not always give optimal solution**.

Depth-limited and iterative deepening search

Depth-limited Search

- ❖ To keep depth-first search from wandering down an infinite path, we can use depth-limited search, a version of depth-first search in which we supply a depth limit, and treat all nodes at depth as if they had no successors
- ❖ The time complexity is $O(b^l)$
- ❖ The space complexity is $O(b^l)$

Note: if we make a poor choice for the l algorithm will fail to reach the solution, making it incomplete again.

Depth-limited and iterative deepening search

❖ Iterative deepening search

- It solves the problem of picking a **good value** for by trying all values: first 0, then 1, then 2, and so on—**until either a solution is found**, or the **depth limited search returns** the failure value **rather than the cutoff value**
- **Iterative deepening combines** many of the benefits of **depth-first** and **breadth-first** search.
 - Like **depth-first search**, its memory requirements are modest: $O(bd)$ when there is a solution, or $O(bm)$ on finite state spaces with no solution.
 - The **time complexity** is $O(bd)$ when there is a solution, or $O(bm)$ when there is none

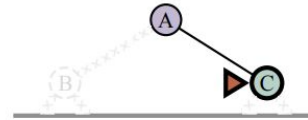
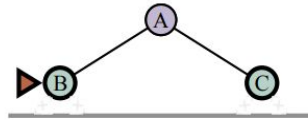
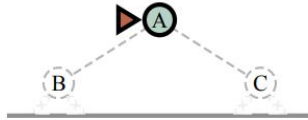
For example: Four iterations of iterative deepening search for goal M on a binary tree, with the depth limit varying from 0 to 3.

Depth-limited and iterative deepening search

limit: 0

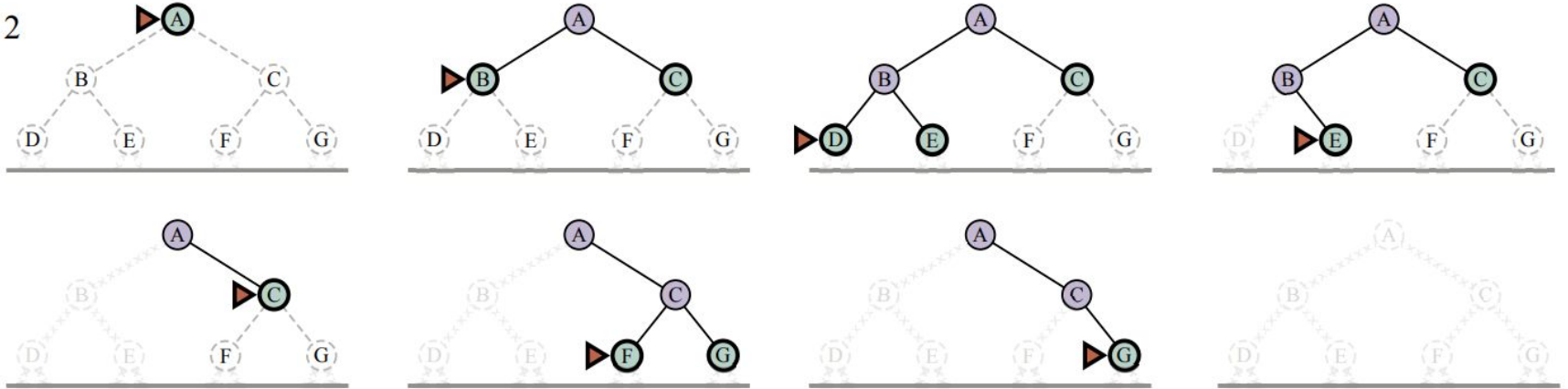


limit: 1



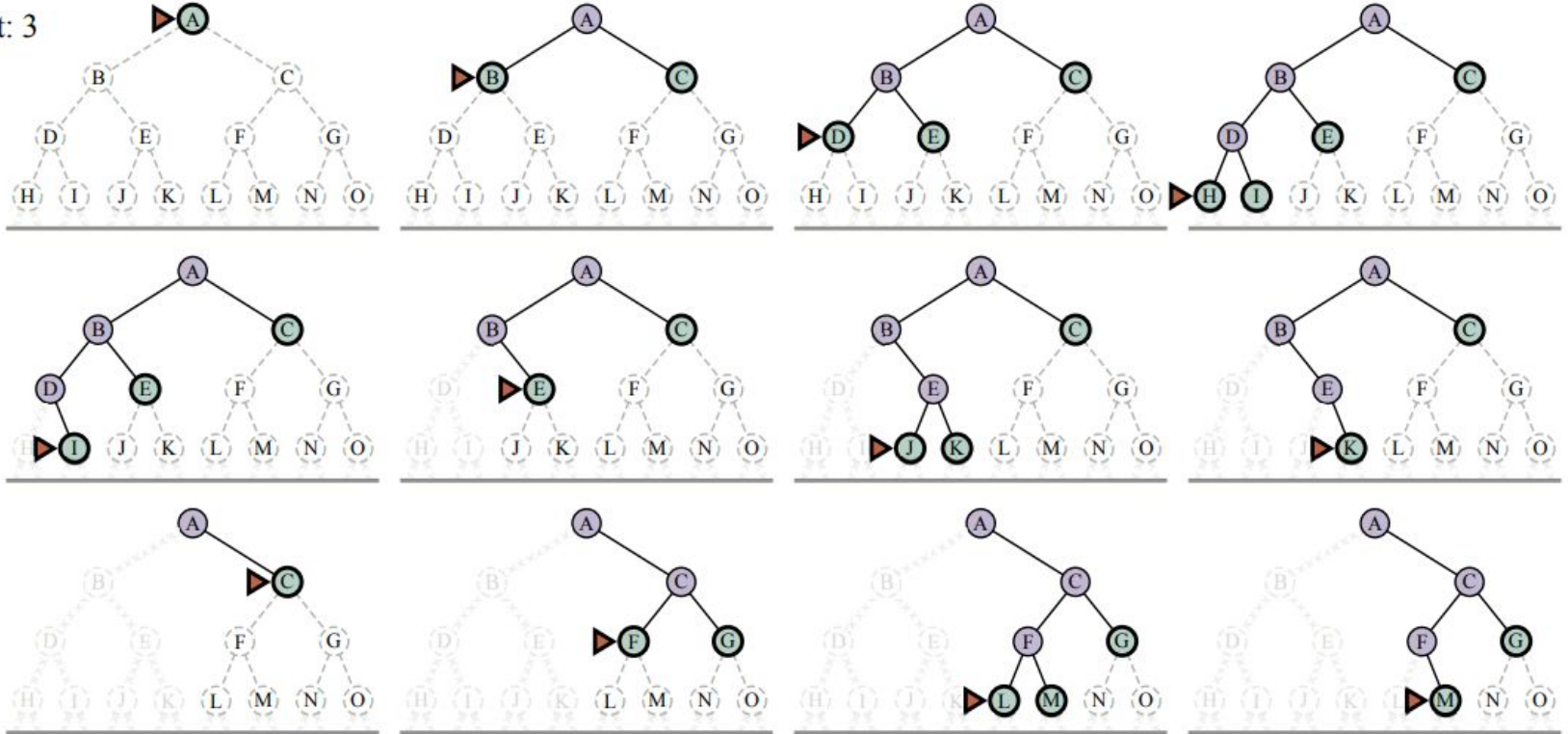
Depth-limited and iterative deepening search

limit: 2



Depth-limited and iterative deepening search

limit: 3



Comparing uninformed search algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹
Optimal cost?	Yes ³	Yes	No	No	Yes ³
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$

Where, b is branching factor,
 d is depth, and

Informed (Heuristic) Search

- ❖ **Best-First Search** is not a single algorithm, but a family of algorithms.
- ❖ It is an "**Informed Search**" strategy that uses an evaluation function $f(n)$ to order the nodes in a Priority Queue.
- ❖ Special type of Best-First Search:
 - Greedy Search
 - A* algorithm

Greedy Best First Search

- ❖ Greedy Best-First Search is an **"Informed Search"** algorithm.
- ❖ It is called **"Greedy"** because it makes the decision that **looks best right now**, without worrying about the **future consequences** or the past costs.
- ❖ Main idea: **"Always move toward the node that seems closest to the Goal."**
- ❖ In Greedy Best-First Search, the node selection is based solely on the heuristic function $h(n)$.
 - $f(n) = h(n)$
 - Where,
 - $f(n)$: The **evaluation score (priority)** of **node n** and
 - $h(n)$: The **Heuristic Function**. This **estimates** the **cost** from **node n** to the **Goal**.
- ❖ At every step, the algorithm expands the node n with the lowest $h(n)$ value.

Greedy Best First Search

❖ Algorithm

- **Initialize:** Put the **Start Node** in the **Priority Queue** (Open List) with its **$h(n)$** value.
- **Loop:** While the **queue is not empty**:
 - **Select:** Remove the **node** with the **lowest $h(n)$** (closest to goal). Call it **Current**.
 - **Goal Test:** If **Current** is the **Goal**, **stop** and **return success**.
 - **Expand:** Generate all **neighbors** of **Current**.
 - **Evaluate:** Calculate **$h(n)$** for each **neighbor** and **add** them to the **queue**.
- **Failure:** If the **queue is empty** and **goal is not found**, return **failure**.

Greedy Best First Search

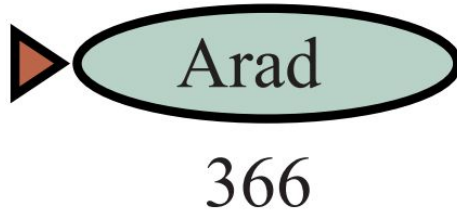
- ❖ Let us see how this works for route-finding problems in **Romania**
- ❖ we use the straight-line distance heuristic, which we will call hSLD
- ❖ If the goal is **Bucharest**, we need to know the straight-line distances to Bucharest, which are shown in Figure.
- ❖ For example, $\text{hSLD}(\text{Arad}) = 366$.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure: Values of hSLD—straight-line distances to Bucharest.

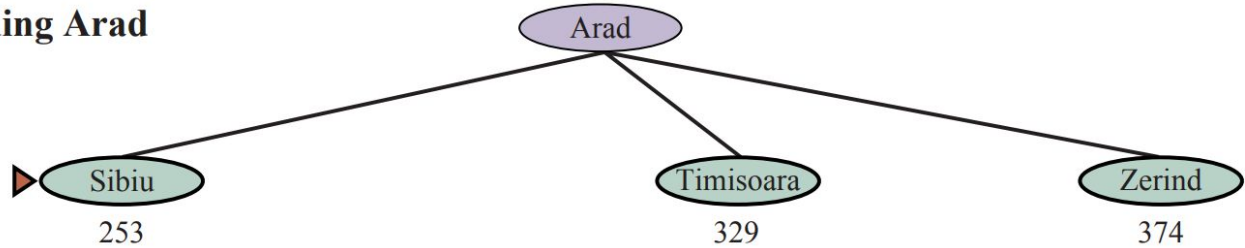
Greedy Best First Search

Initial State



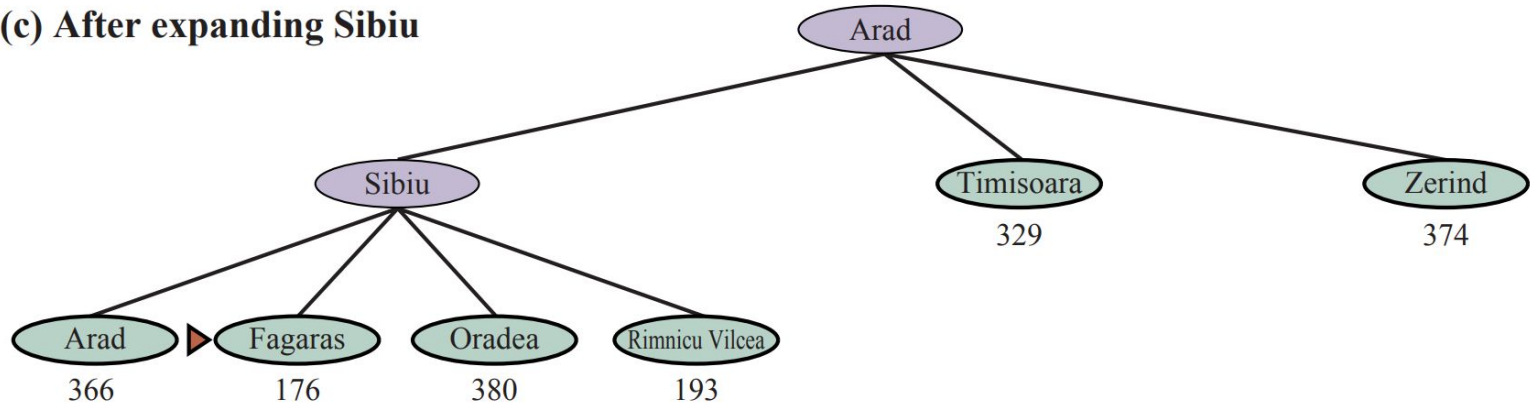
Greedy Best First Search

(b) After expanding Arad



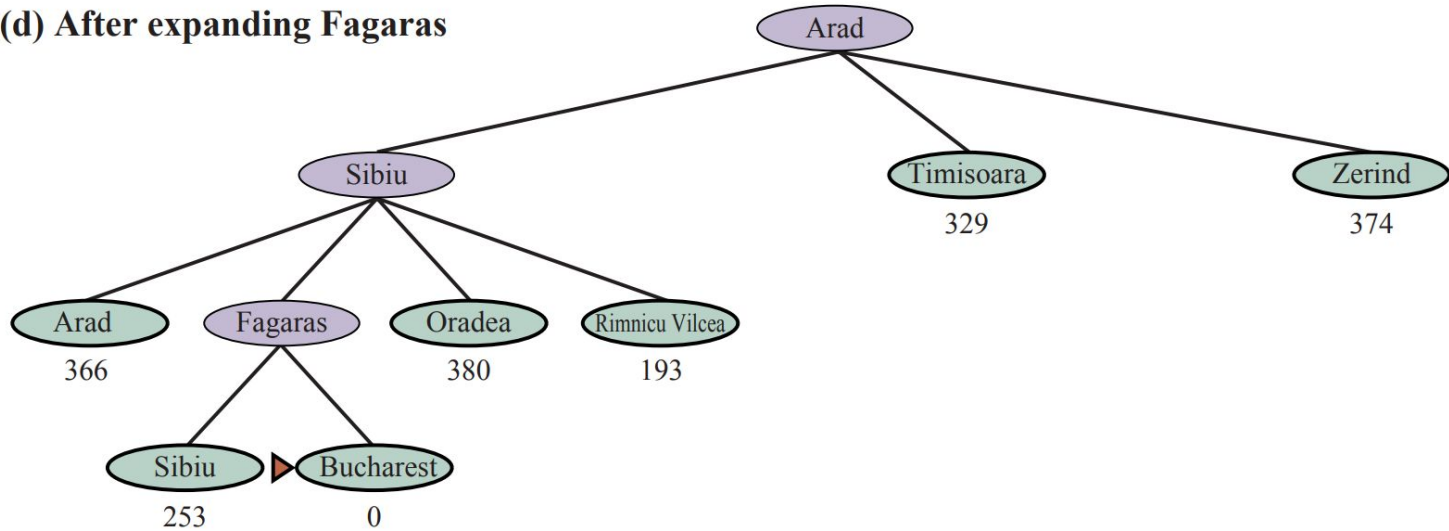
Greedy Best First Search

(c) After expanding Sibiu



Greedy Best First Search

(d) After expanding Fagaras



Greedy Best First Search: Evaluation Parameters

Complete : NO (Loop)

Time Complexity : $O(b^m)$, but good heuristic can give improvement

Space Complexity: $O(b^m)$ as keeps all node in memory

Optimal: NO

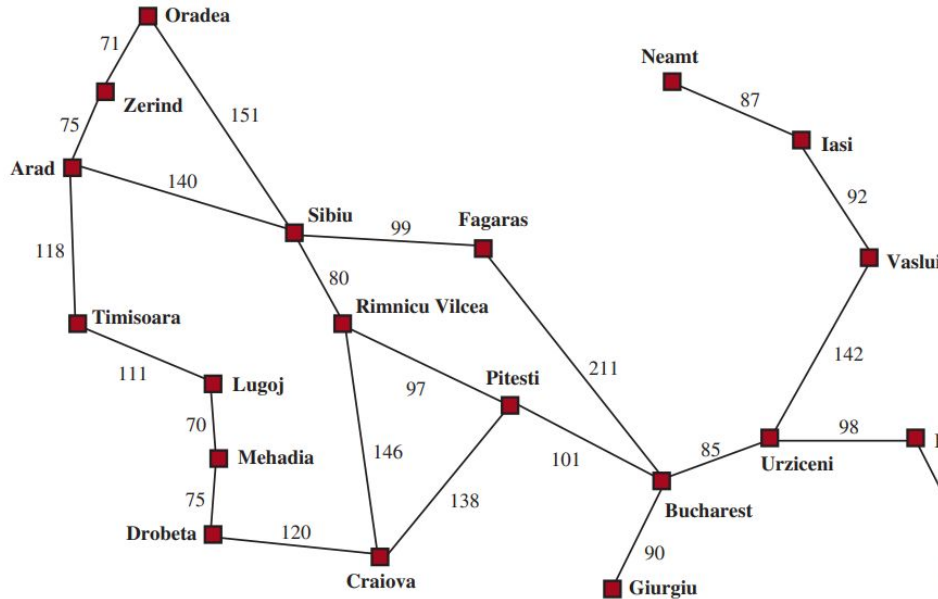
A* Search Algorithm

- ❖ The most common informed search algorithm is **A* search** (pronounced "A-star search")
- ❖ The "Smart" version of the search. Unlike Greedy Search (**which ignores the past**), **A* considers** both the **distance** you have **already traveled** and the estimated distance left.
- ❖ Formula

$$f(n) = g(n) + h(n)$$

- $g(n)$ (Actual Cost): "How many kilometers have I already driven from the Start to get here?" (Road Distance).
- $h(n)$ (Heuristic): "How far is the goal from here?" (Straight-Line Distance).
- $f(n)$ (Total Estimated Cost): "How long will the entire trip take if I go this way?"

A* Search Algorithm

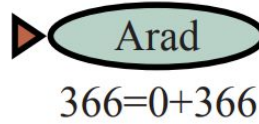


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure: Values of hSLD—straight-line distances to Bucharest.

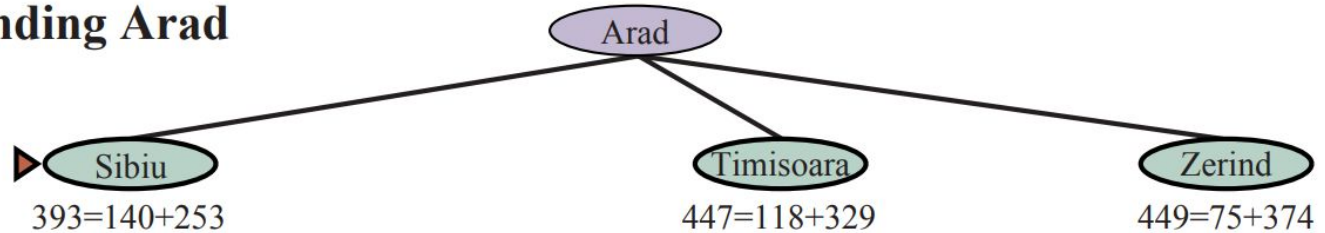
A* Search Algorithm: Working

Initial State



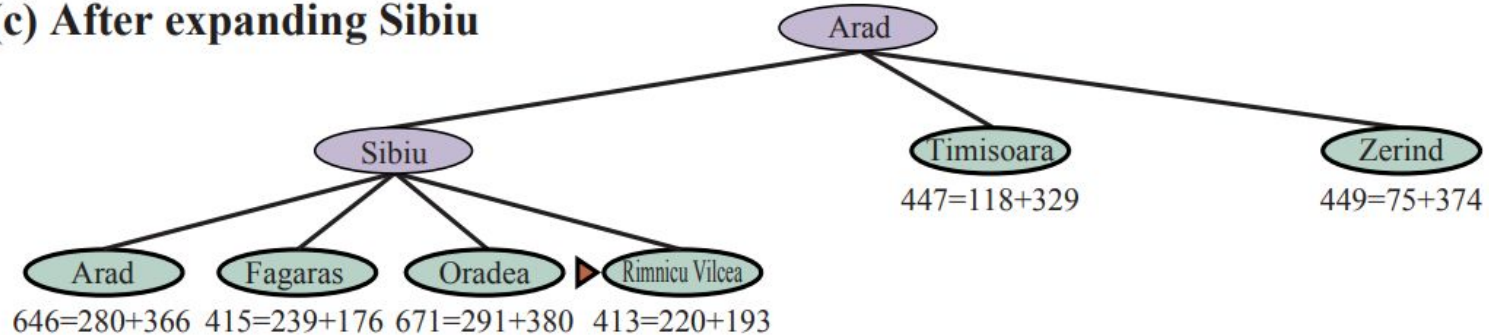
A* Search Algorithm: Working

(b) After expanding Arad



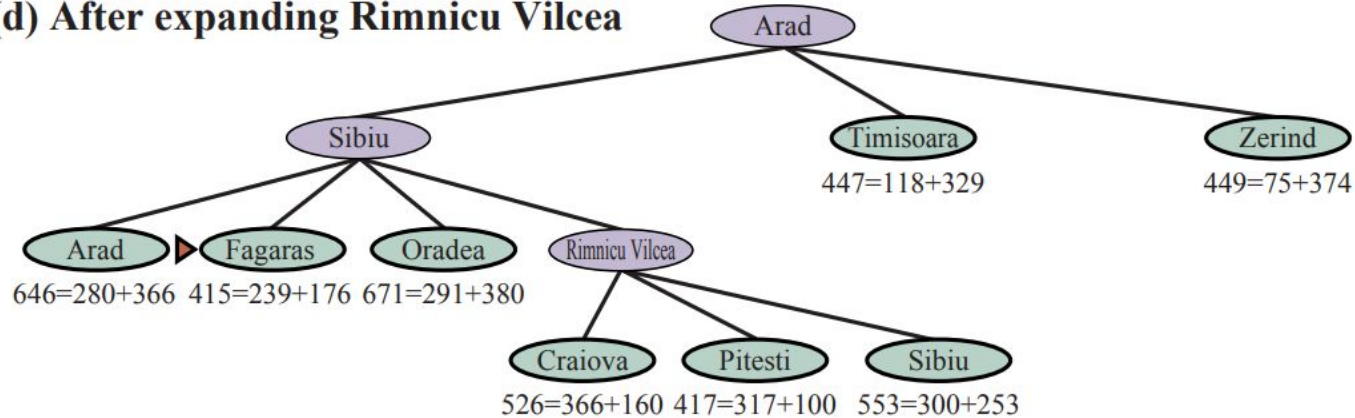
A* Search Algorithm: Working

(c) After expanding Sibiu



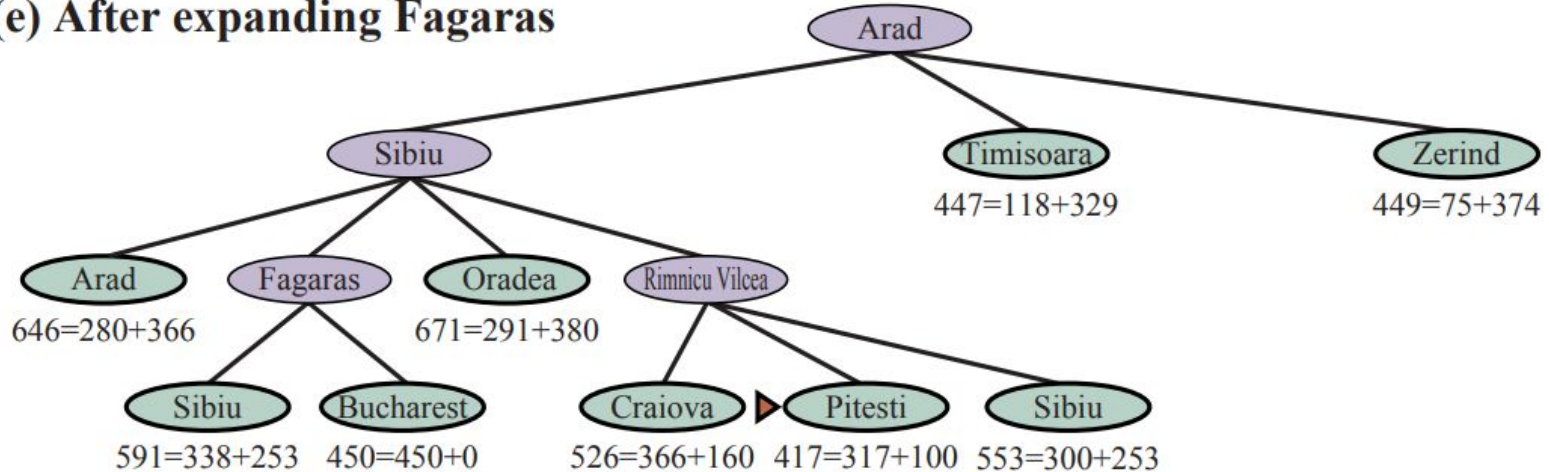
A* Search Algorithm: Working

(d) After expanding Rimnicu Vilcea

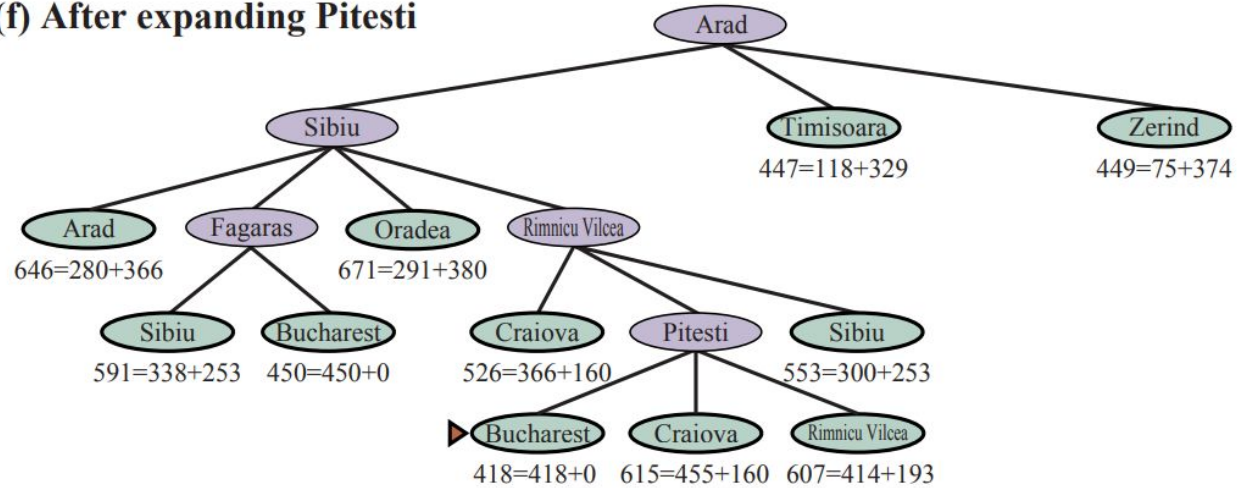


A* Search Algorithm: Working

(e) After expanding Fagaras



(f) After expanding Pitesti



Note:

- ❖ **Bucharest** first **appears** on the **frontier** at step (e), but it is **not selected** for expansion (and thus not detected as a solution) **because at $f=450$,**
- ❖ It is **not the lowest-cost node** on the frontier—that would be **Pitesti, at $f=417$.**
- ❖ **Why?** There might be **a solution** through **Pitesti** whose **cost** is as **low as 417,**
- ❖ so the **algorithm** will not settle for a solution that **costs 450.**
- ❖ At step (f), a different path to **Bucharest** is **now the lowest-cost node,**
- ❖ At so it is **selected** and **detected** as the optimal solution.

A* Search Algorithm: Evaluation

- ❖ A* is admissible and considers **fewer nodes** than any other admissible search algorithm with the same heuristic.
- ❖ This is because **A*** uses an "**optimistic**" estimate of the **cost of a path** through every node that it considers—optimistic in that the true cost of a path through that node to the goal will be at least as great as the estimate.
- ❖ But, critically, as far as A* "knows", that optimistic estimate might be achievable.

Adversarial Search Techniques

- ❖ Adversarial search techniques are used in environments where two or more agents (often referred to as "players") compete with each other, and each player tries to maximize their own benefit while minimizing the opponent's benefit.
- ❖ These techniques are particularly common in game-playing scenarios like chess, checkers, or tic-tac-toe

Minimax Algorithm

- ❖ **Mini-max** algorithm is a **recursive** or **backtracking** algorithm which is used in **decision-making** and **game theory**. It provides an optimal move for the player assuming that opponent is also playing optimally.
- ❖ Mini-Max algorithm uses **recursion** to **search** through the **game-tree**.
- ❖ Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various two-players game. This Algorithm computes the minimax decision for the current state.
- ❖ In this algorithm **two players** play the game, one is called **MAX** and other is called **MIN**.
- ❖ Both the players fight it as the **opponent player** gets the **minimum benefit** while they get the **maximum benefit**.

The minimax search algorithm

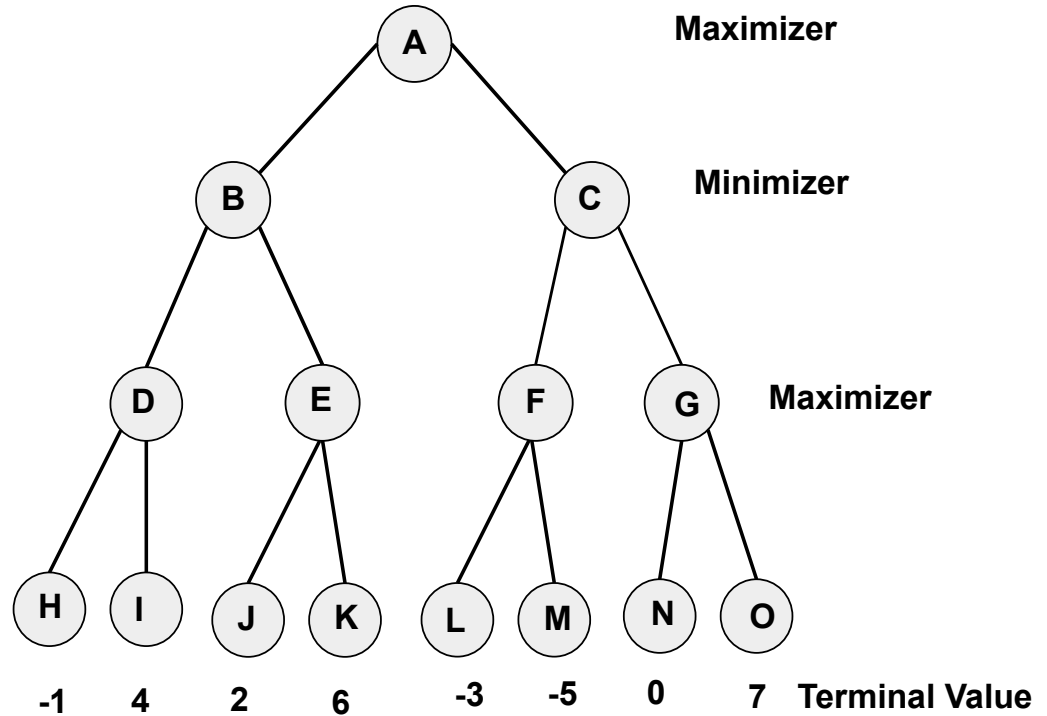
Algorithm

1. Generate game tree completely
2. Determine utility of each terminal state
3. Propagate the utility values upward in tree by applying MIN and MAX operators on nodes in the current level
4. At the root node use minimum decision to select move max (of the main) utility value.

Note: step (2) and (3) in the algorithm assumes that the opponent will play perfectly

The minimax search algorithm—Step 1

Check all the terminal values



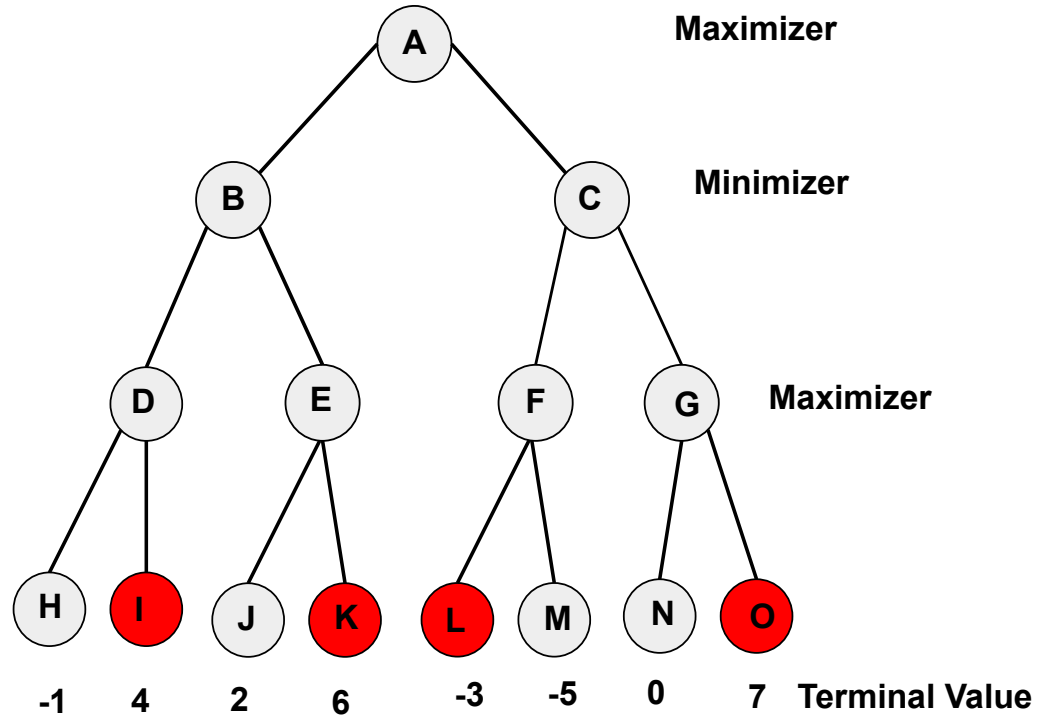
The minimax search algorithm—Step 2

For node D, $\max(-1, 4) = 4$

For Node E, $\max(2, 6) = 6$

ForNode F, $\max(-3, -5) = -3$

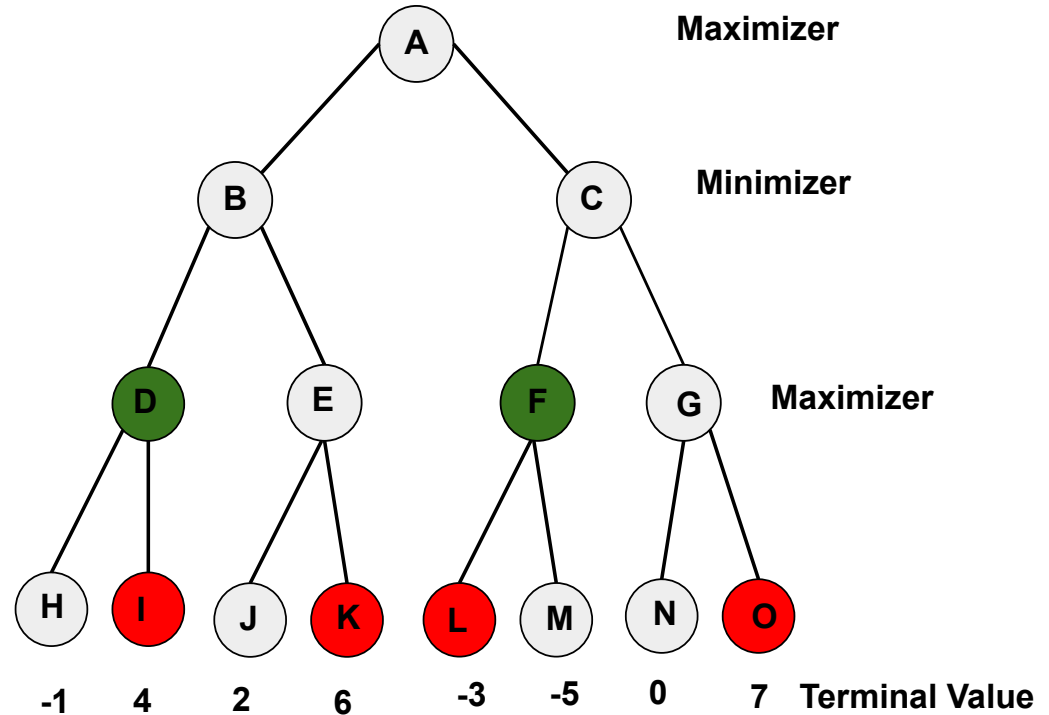
For node G, $\max(0, 7) = 7$



The minimax search algorithm—Step 3

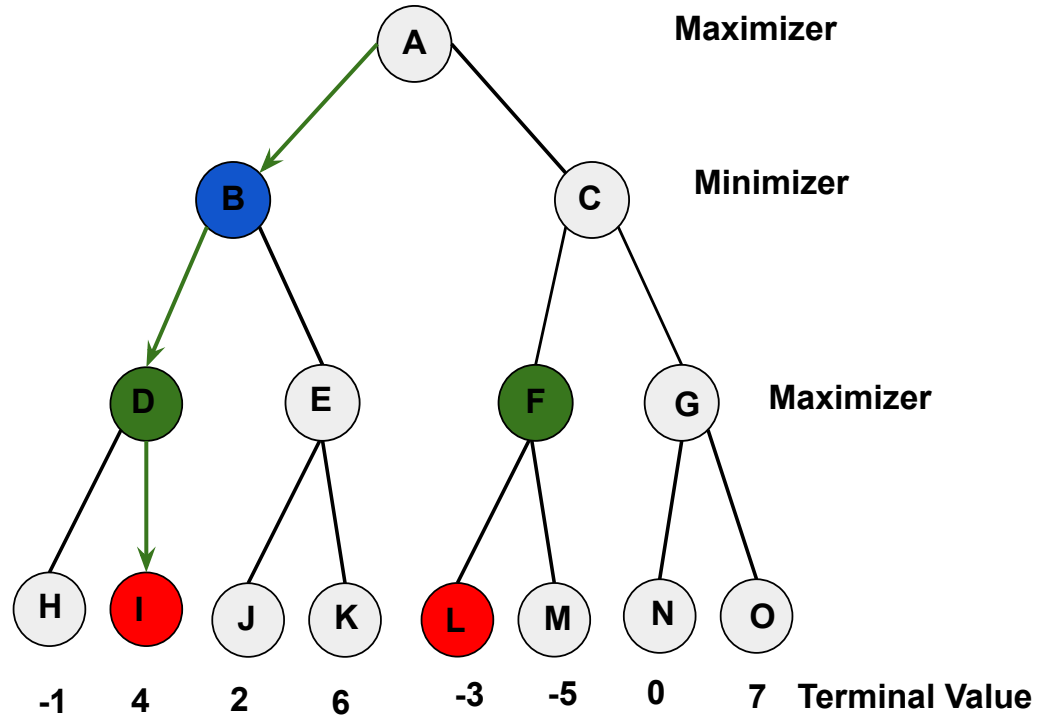
For node B, $\min(4, 6) = 4$

For node C, $\min(-3, 7) = -3$



The minimax search algorithm—Step 4

- For node A, $\max(4, -3) = 4$



The minimax search algorithm: Performance Measures

Complete- Min-Max algorithm is **Complete**. It will definitely find a **solution** (if exist), in the finite search tree.

Optimal- Min-Max algorithm is optimal if **both opponents are playing optimally**.

Time complexity- $O(b^m)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.

Space Complexity- $O(b^m)$.

The minimax search algorithm: Limitation

- ❖ The main drawback of the minimax algorithm is that it gets **really slow** for complex games such as **Chess**, go, etc.
- ❖ This type of games has a huge branching factor, and the player has lots of choices to decide.
- ❖ This limitation of the minimax algorithm can be improved from alpha-beta pruning.

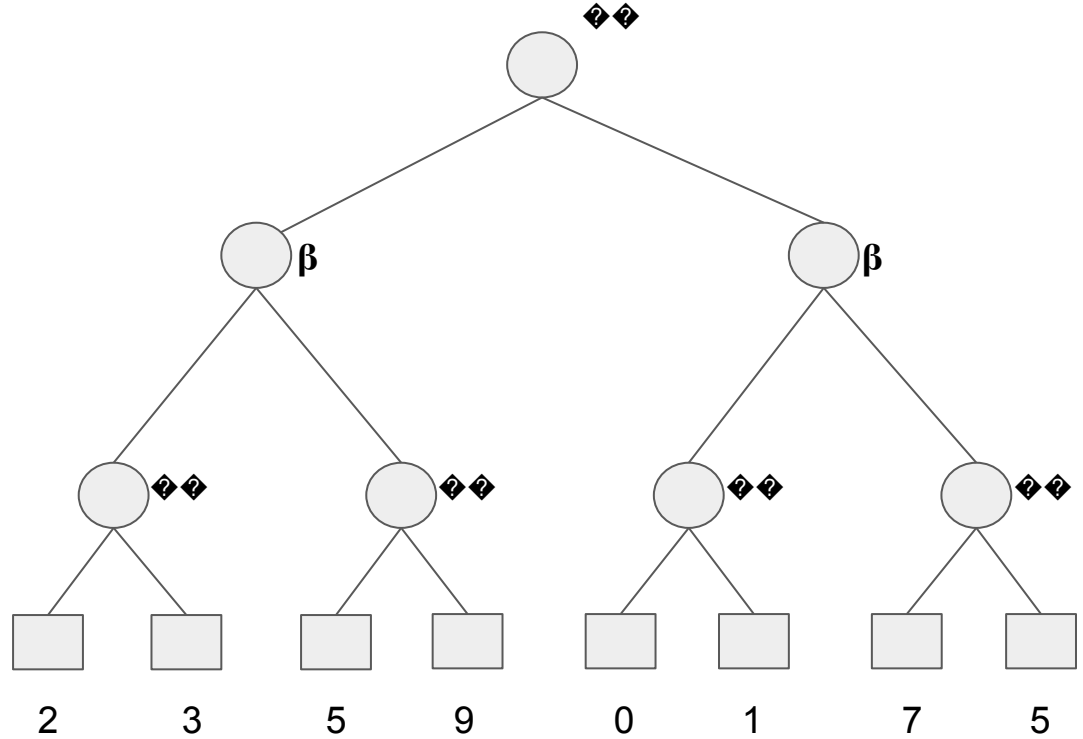
Alpha-Beta Pruning

- ❖ Alpha-Beta Pruning is an optimization technique for the Minimax algorithm.
- ❖ **The Problem with Minimax:** It checks every branch, even the bad ones. It is slow.
- ❖ It stops checking a branch (prunes it) as soon as it realizes that the branch is worse than a move found earlier.
- ❖ It returns the exact same answer as Minimax, but much faster.
- ❖ Alpha-beta pruning can be applied to trees of any depth, and it is often possible to prune entire sub-trees rather than just leaves.
- ❖ **α (Alpha):** The best (highest) value MAX has found so far along the path. MAX updates Alpha.
- ❖ **β (Beta):** The best (lowest) value MIN has found so far along the path. MIN updates Beta.
- ❖ The Pruning Condition:
 - If $\alpha \geq \beta$, stop looking! (Cut the branch).

Alpha-beta pruning

Depth first

Terminal value is 2



Alpha-beta pruning

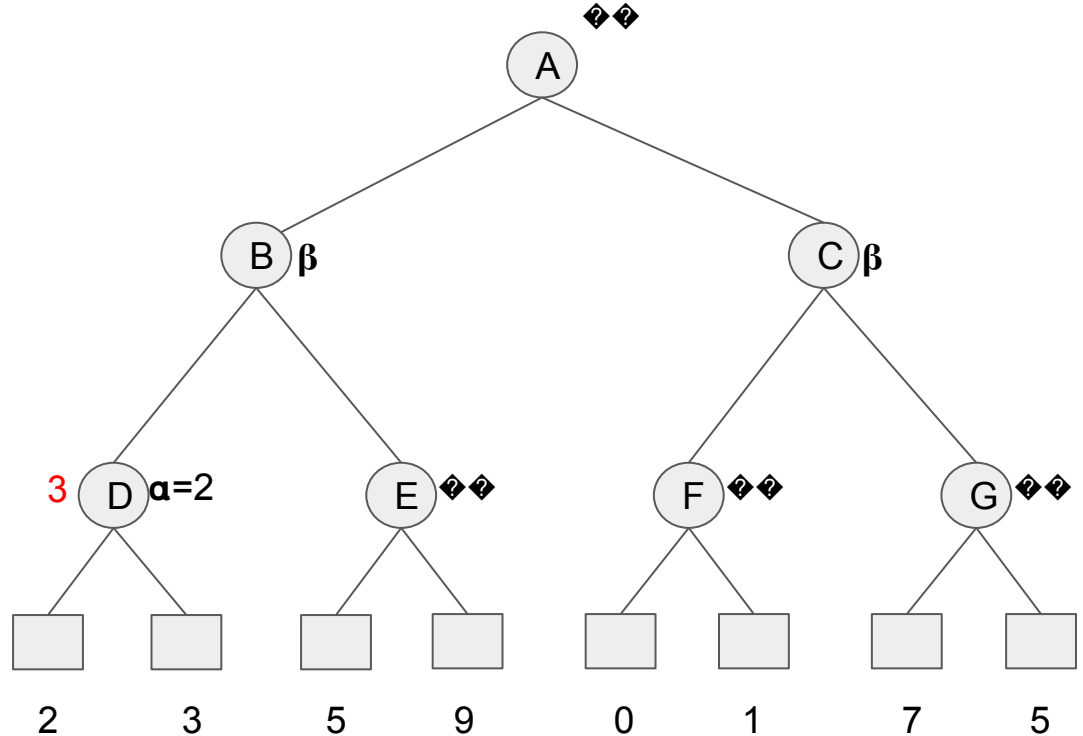
At Node D (MAX),

Set value of $\alpha=2$

The value of node D will be ≥ 2

Go to next node, i.e. terminal value 3

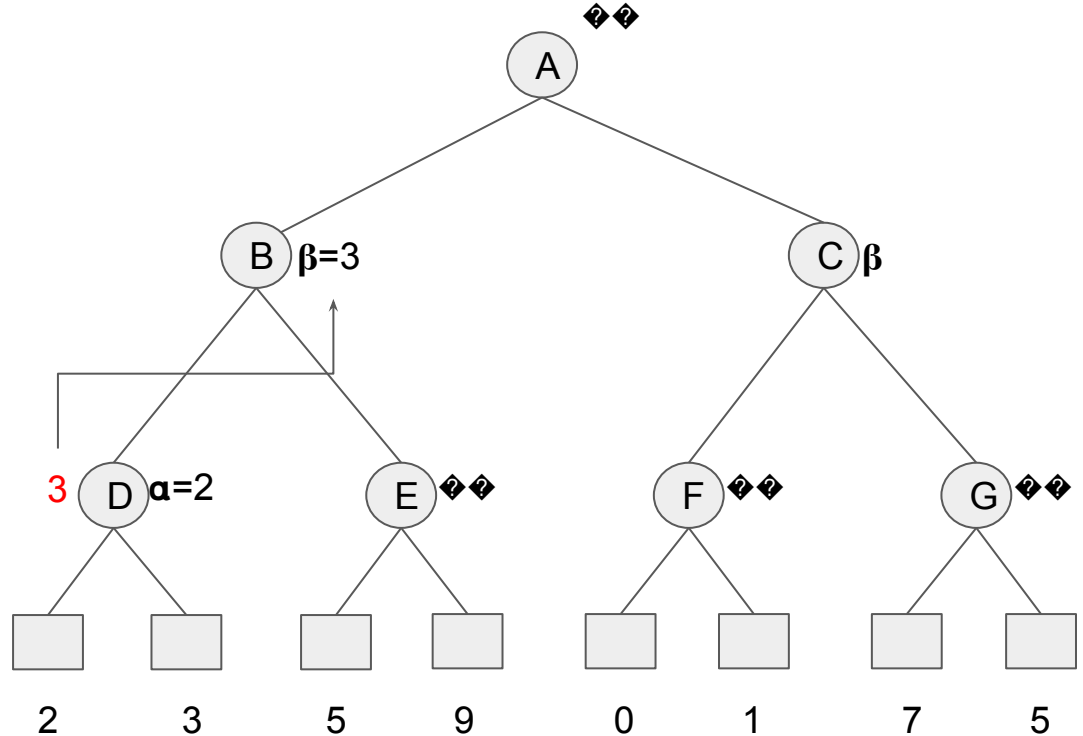
Therefore, the value of node D = $\text{MAX}(2, 3) = 3$



Alpha-beta pruning

At Node D value is now fixed;
take its value above to node B,
i.e., $\beta=3$

We know now that the
minimum value of node B will
be ≤ 3



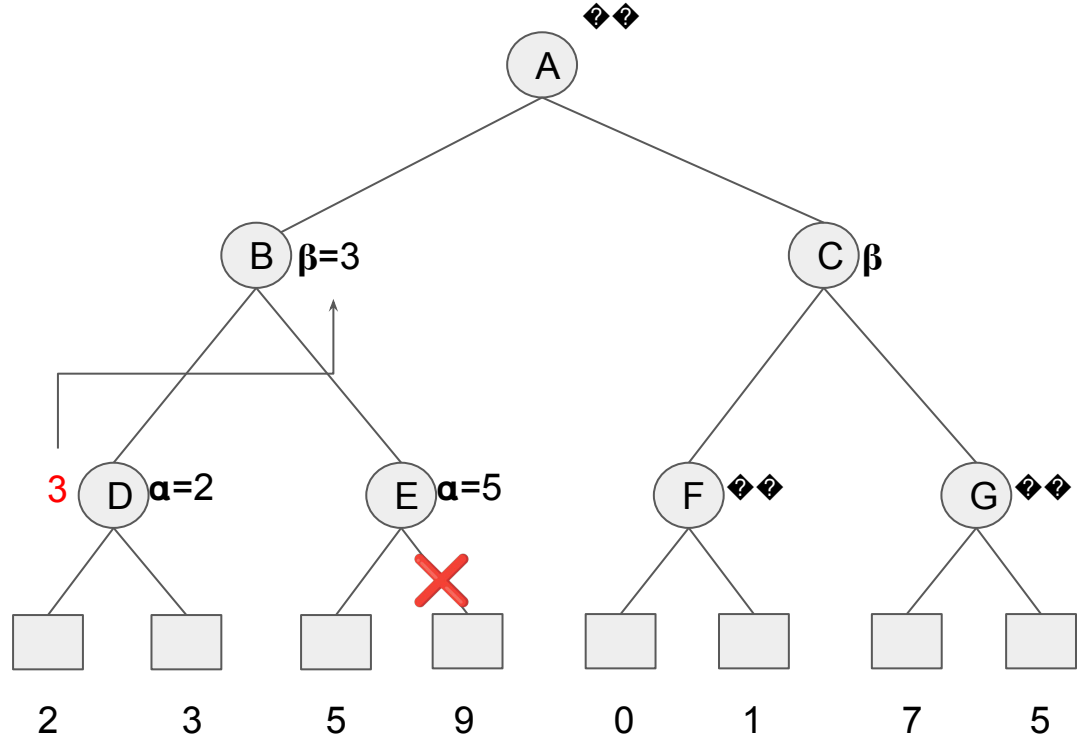
Alpha-beta pruning

We now go to node E and then to terminal node with value 5

Now the value of $\alpha=5$

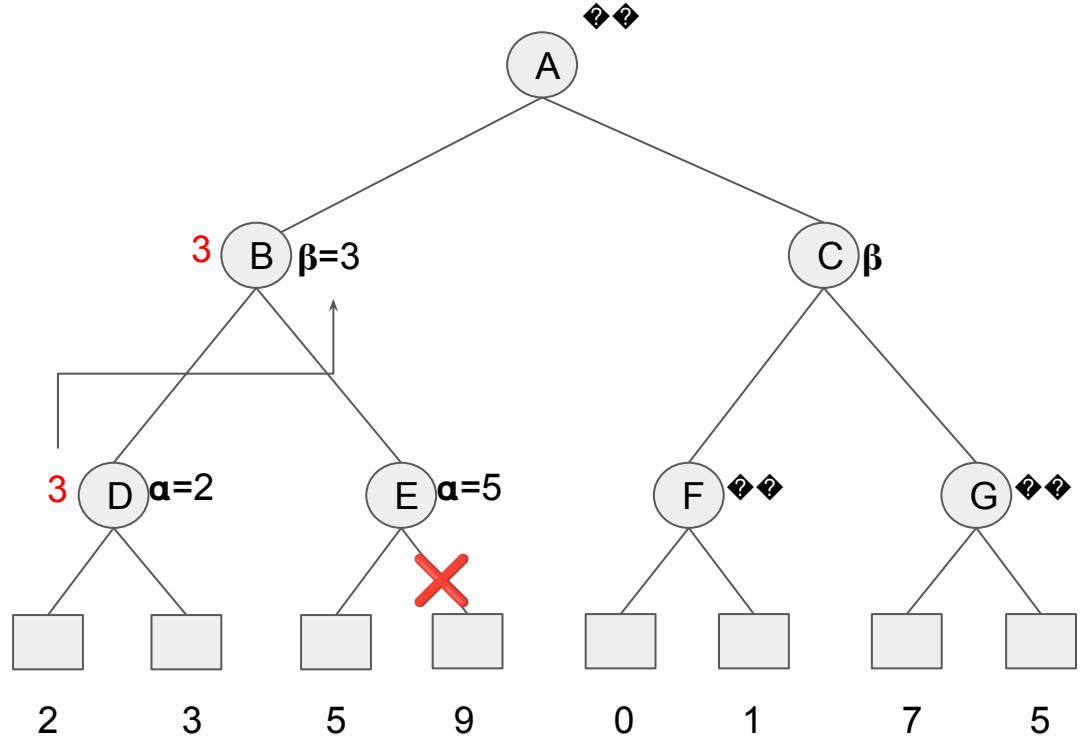
Note: Value of node $E \geq 5$ but the value of node $B \leq 3$, i.e., $\alpha > \beta$

Therefore now the right of E will be **pruned**, i.e., **no need to explore terminal node with value 9**



Alpha-beta pruning

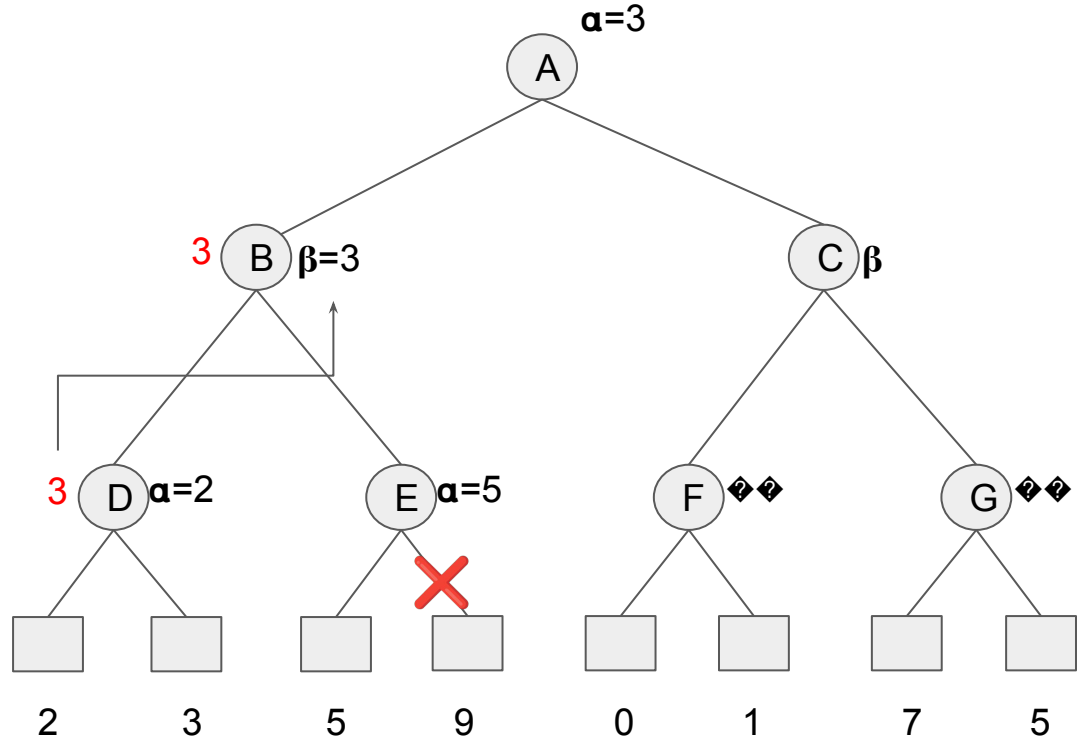
Value of node B = 3



Alpha-beta pruning

Node B is fixed take its value of upper node for $\alpha=3$

The MAX value of node will be ≥ 3

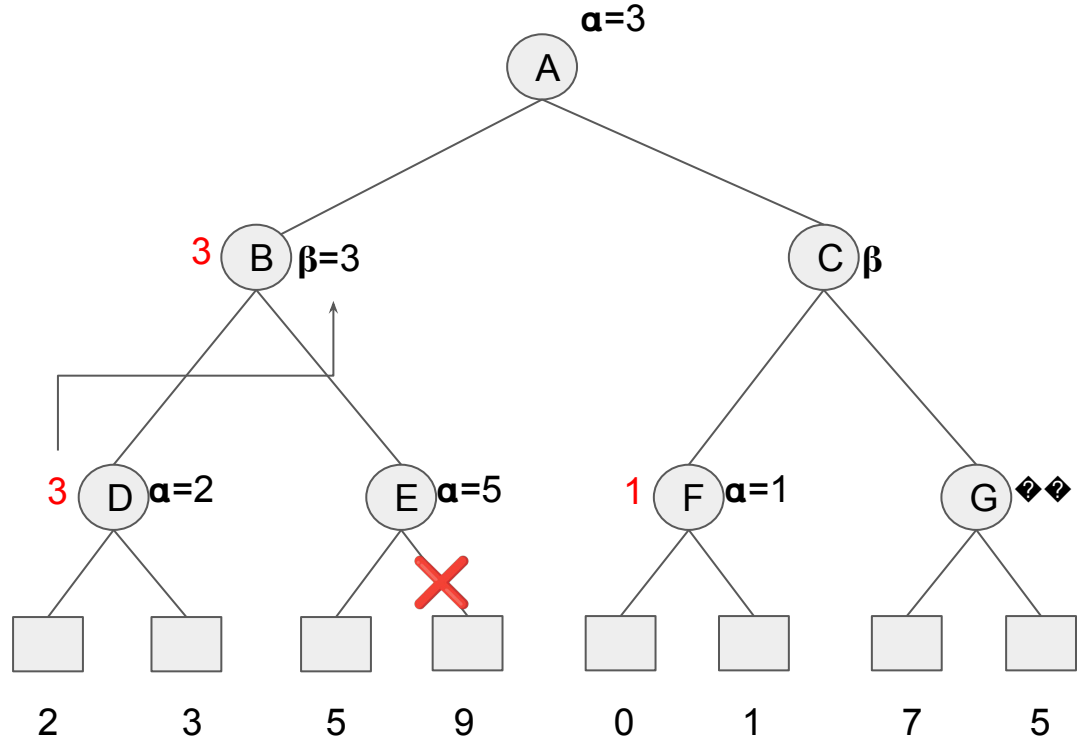


Alpha-beta pruning

Terminal node 0

Node F, $\alpha=0$

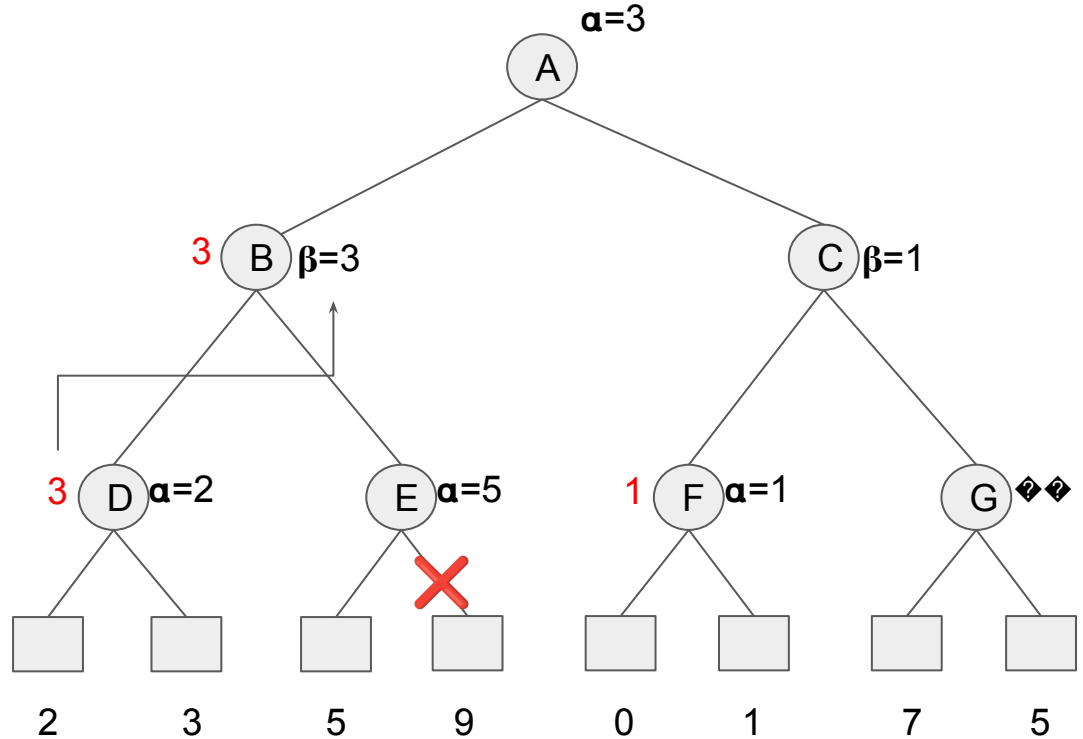
Value of Node F = $\text{MAX}(0, 1) = 1$



Alpha-beta pruning

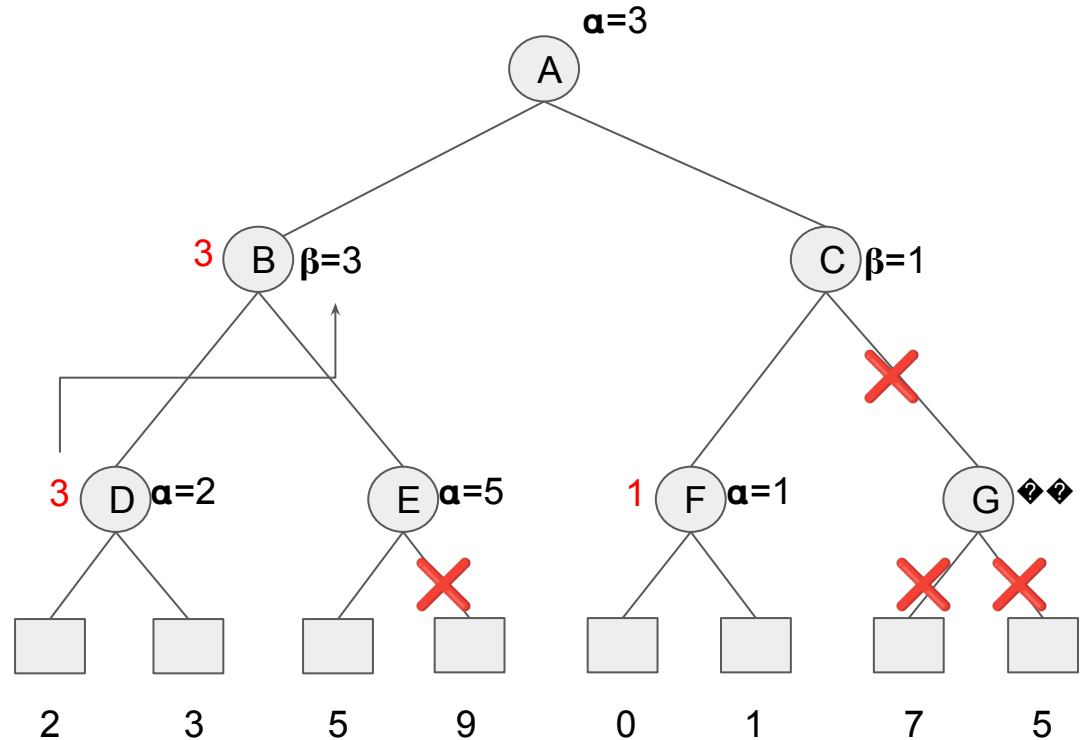
Node C, $\beta=1$

Decision: The value of node C will be ≤ 1



Alpha-beta pruning

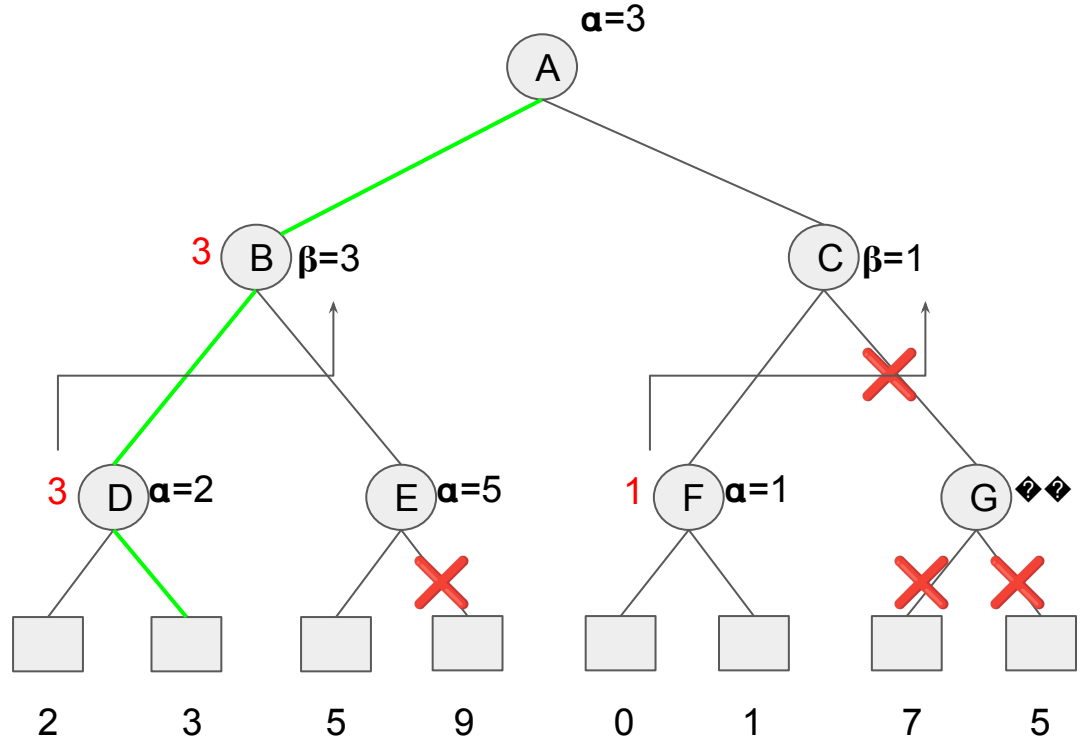
Note: Value of node **A** ≥ 3 but the value of node **C** ≤ 1 or $\alpha > \beta$. Rest of the path is pruned



Alpha-beta pruning

C now returns the value of 1 to A here the best value for A is $\max(3, 1) = 3$.

Hence the **optimal value** for the **maximizer** is **3** for this example.



Alpha-beta pruning

Conclusion:

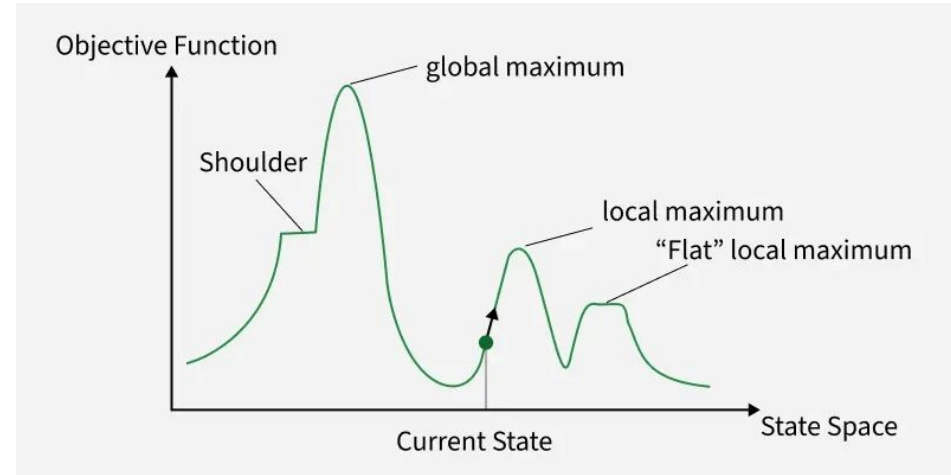
Most of the nodes are not
explored

Local Search

- ❖ Unlike **A*** or **BFS**, which care about the **path to the goal**, **Local Search algorithms** do **not care about the path**. They only care about **finding the best final state (Optimization)**.
- ❖ They are useful for big or complex problems where checking every possible option isn't practical.
 - It focus only on the current solution and the ones directly related to it rather than looking everywhere.
 - Ideal for real-world tasks like puzzles, timetables or route finding.
- ❖ **Types**
 - **Hill-Climbing Search Algorithm**
 - **Simulated Annealing**
 - **Genetic Algorithms**

Hill Climbing Search

- ❖ Hill Climbing is a simple "**Greedy**" local search algorithm.
- ❖ It constantly moves in the direction of increasing value (uphill) to find the peak.
- ❖ It terminates when it reaches a peak where no neighbor has a higher value.
- ❖ Algorithm:
 - **Initial State:** Start with an arbitrary or random solution (initial state).
 - **Neighboring States:** Identify neighboring states of the current solution by making small adjustments (mutations or tweaks).
 - **Move to Neighbor:** If one of the neighboring states offers a better solution (according to some evaluation function), move to this new state.
 - **Termination:** Repeat this process until no neighboring state is better than the current one. At this point, we have reached a local maximum or minimum.



Hill Climbing Search

❖ Types :

➤ **Simple Hill Climbing:**

- Take the first step that looks better.
- The algorithm evaluates each neighbouring node one by one and selects the first node that offers an improvement over the current one.

➤ **Steepest-Ascent Hill Climbing**

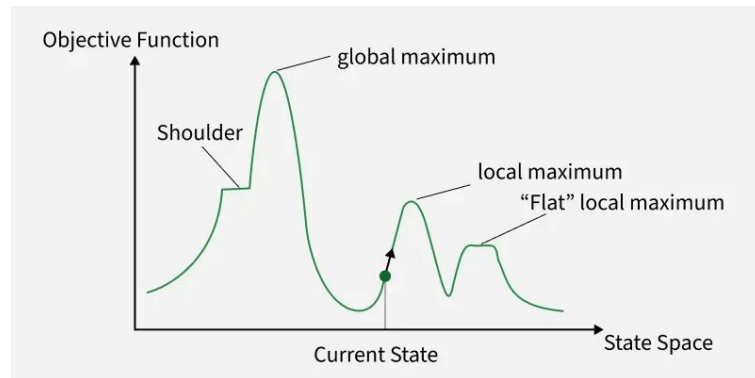
- "Look at all options, then take the best one."
- Makes the optimal decision at each individual step.
- Slower (because it must check everything). Still gets stuck in Local Maxima.

➤ **Stochastic Hill Climbing**

- Pick a neighbor at random. If it's better, take it."
- Why use it? The randomness helps it avoid getting stuck in simple traps or plateaus where Steepest-Ascent might get confused.

Hill Climbing Search

- ❖ **Local Maximum:** A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
- ❖ **Global Maximum:** The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
- ❖ **Plateau/Flat Local Maximum:** A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.
- ❖ **Ridge:** A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
- ❖ **Current State:** The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
- ❖ **Shoulder:** A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better solutions if it continues searching beyond the plateau.



Simulated Annealing

- ❖ Simulated Annealing is an optimization algorithm designed to search for an optimal or near-optimal solution in a large solution space.
- ❖ The process of **annealing** in **metallurgy**, where a material is heated and then slowly cooled to remove defects and achieve a stable crystalline structure
- ❖ In Simulated Annealing, the "**heat**" corresponds to the **degree of randomness** in the search process, which decreases **over time** (cooling schedule) to refine **the solution**.
- ❖ It escaping these **local minima** by introducing controlled **randomness in its search**, allowing for a more thorough exploration of the solution space.
- ❖ **Why is it better than Hill Climbing?**
 - **Hill Climbing:** "I am on a small hill. To get to the big mountain, I have to go down into the valley first. But I refuse to go down. So, I am stuck here forever." (Stuck in Local Maximum).
 - **Simulated Annealing:** "I am on a small hill. The temperature is high, so I am allowed to make a 'mistake' and walk down into the valley. This allows me to cross over and climb the huge mountain on the other side." (Finds Global Maximum).

Simulated Annealing: Algorithm

- ❖ **Initialization:**
 - Start with a random first solution and a high temperature. The high temperature makes it easy to accept worse options at the beginning.
- ❖ **Neighborhood Search:**
 - Make a small change to your current solution to create a new, nearby solution.
- ❖ **Objective Function Evaluation:**
 - Check the quality of the new solution. If it is better than your current one, accept it immediately.
- ❖ **Acceptance Probability:**
 - If the new solution is worse, you might still accept it. The odds of accepting a worse solution depend on how bad it is and how hot the temperature is.
- ❖ **Cooling Schedule:**
 - Lower the temperature slightly after each step. This cooling determines how fast you stop taking risks and start settling on a final answer.
- ❖ **Termination:**
 - Stop the process when the temperature gets too cold to make any more changes, or when you run out of steps.

Simulated Annealing: Advantages

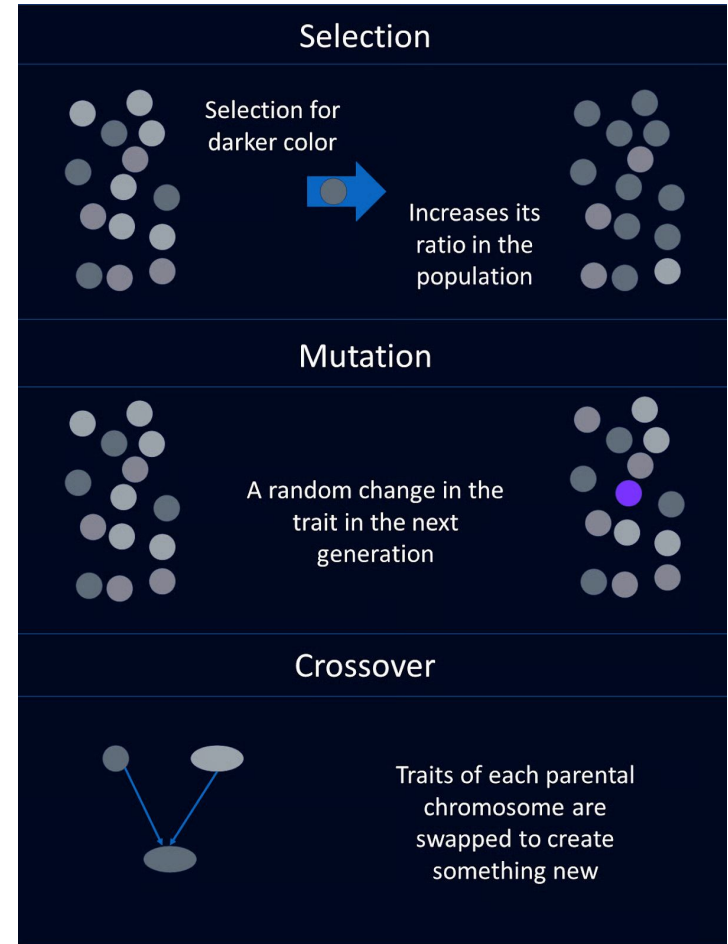
- ❖ **Global Optimization (Finds the Real Best):** It is capable of finding the Global Optimum even in a bumpy, complex search space. It doesn't get stuck in small local peaks easily.
- ❖ **Robustness (Works on Messy Problems):** It works well on problems where the "landscape" is rough, noisy, or has many peaks and valleys. It is not easily confused by small bumps in the data.
- ❖ **Versatility (Universal Key):** You can use it for almost any type of optimization problem (scheduling, routing, circuit design). You don't need to know advanced calculus or gradients; you just need to know how to calculate the "Score" (Value) of a state.
- ❖ **Statistically Guaranteed:** If you lower the temperature slowly enough (a slow cooling schedule), it is mathematically proven to converge to the optimal solution. (Note: In reality, this might take a long time, but the guarantee exists).
- ❖ **Ease of Implementation:** The algorithm is surprisingly simple to code. It doesn't require complex memory structures (like keeping a huge tree in BFS). It only keeps track of one current state.

Simulated Annealing: Limitations

- ❖ **Parameter Sensitivity:** The performance of Simulated Annealing is highly dependent on the choice of parameters, particularly the initial temperature and cooling schedule.
- ❖ **Computational Time:** Since Simulated Annealing requires many iterations, it can be computationally expensive, especially for large problems.
- ❖ **Slow Convergence:** The convergence rate is generally slower than more deterministic methods like gradient-based optimization.

Genetic Algorithm

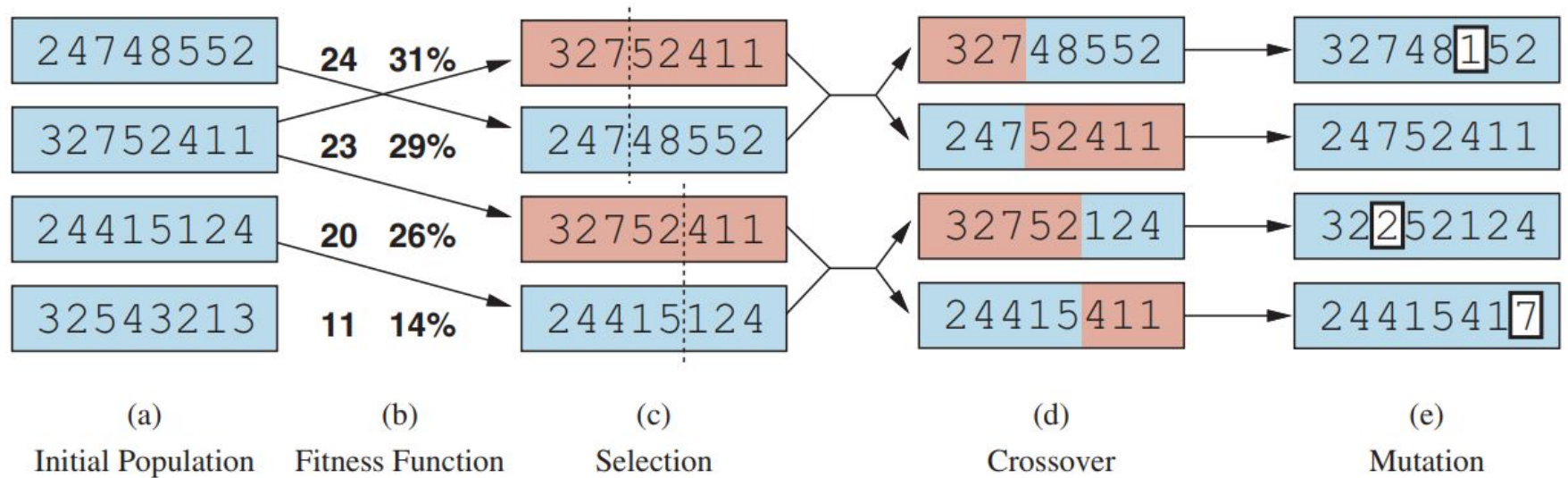
- ❖ A **Genetic Algorithm** is an optimization search technique inspired by Charles Darwin's Theory of Natural Evolution.
- ❖ **Genetic algorithms** are commonly used to generate high-quality solutions to **optimization** and **search problems** via biologically inspired operators such as **selection**, **crossover**, and **mutation**.
- ❖ **Key Terminology:**
 - **Population:** The set of all current solutions (individuals).
 - **Chromosome:** A single solution (usually represented as a string of 1s and 0s, like DNA).
 - **Gene:** A specific bit (0 or 1) inside the chromosome.
 - **Fitness Function:** A score given to each solution. Higher score = Better solution (More likely to reproduce).



Genetic Algorithm

- ❖ **Step 1: Initialization**
 - First, we generate an initial population of random individuals. This step creates a diverse set of potential solutions to start the algorithm.
- ❖ **Step 2: Evaluation**
 - Next, we need to calculate the fitness of each individual in the population. Here we use the fitness function to evaluate how good each solution is.
- ❖ **Step 3: Selection**
 - Using the selection criteria, we select individuals for reproduction based on their fitness. This step determines which individuals will be parents.
- ❖ **Step 4: Crossover**
 - Crossover comes next. By combining the genetic material of selected parents, we apply crossover techniques to generate new solutions or offspring.
- ❖ **Step 5: Mutation**
 - To maintain diversity in our population, we need to introduce random mutations in the offspring.
- ❖ **Step 6: Replacement**
 - We next replace some or all of the old population with the new offspring, by determining which individuals move on to the next generation.
- ❖ **Step 7: Repeat**
 - The previous steps 2-6 are looped over for a set number of generations or until a termination condition is met. This loop allows the population to evolve over time, hopefully resulting in a good solution.

Genetic Algorithm: Numerical



Genetic Algorithm : Example

Here's a **simple example** of using a **Genetic Algorithm (GA)** to optimize the equation $f(x) = 25x - 5x^2$ for integer values of x in the range 0 to 31. We'll use an initial population size of $n = 4$.

Step 1: Define the Problem

- **Objective:** Maximize $f(x) = 25x - 5x^2$.
- **Domain:** x is an integer between 0 and 31.
- **Population Size:** $n = 4$.

Genetic Algorithm : Example

Step 2: Encode the Solution

- Represent x as a **5-bit binary string** (since $2^5 = 32$ covers the range 0 to 31).
- Example:
 - $x = 5 \rightarrow$ Binary: 00101
 - $x = 12 \rightarrow$ Binary: 01100

Genetic Algorithm : Example

Step 3: Initialize Population

Randomly generate 4 individuals (binary strings) and calculate their fitness ($f(x)$):

Individual	Binary String	x	$f(x) = 25x - 5x^2$	Fitness
1	01010	10	$25(10) - 5(10)^2 = -250$	-250
2	00101	5	$25(5) - 5(5)^2 = 0$	0
3	10011	19	$25(19) - 5(19)^2 = -760$	-760
4	01100	12	$25(12) - 5(12)^2 = -420$	-420

Genetic Algorithm : Example

Step 4: Selection

Select individuals for reproduction based on their fitness. Use **roulette wheel selection** or **tournament selection**.

- Higher fitness values are better, but since our fitness values are negative, we can use the absolute value or shift the values.
- For simplicity, let's select the top 2 individuals:
 - Individual 2 ($x = 5$, Fitness = 0)
 - Individual 1 ($x = 10$, Fitness = -250)

Genetic Algorithm : Example

Step 5: Crossover

Perform crossover to create offspring. Let's use **single-point crossover** at a random point.

- Parent 1: 00101 ($x = 5$)
- Parent 2: 01010 ($x = 10$)

Crossover point: After the 2nd bit

- Offspring 1: 00010 ($x = 2$)
- Offspring 2: 01101 ($x = 13$)

Genetic Algorithm : Example

Step 6: Mutation

Introduce small random changes to the offspring. Let's flip one bit in Offspring 2:

- Offspring 2 before mutation: 01101 ($x = 13$)
- Offspring 2 after mutation: 01111 ($x = 15$)

Genetic Algorithm : Example

Step 7: New Population

Replace the old population with the new individuals:

Individual	Binary String	x	$f(x) = 25x - 5x^2$	Fitness
1	00101	5	$25(5) - 5(5)^2 = 0$	0
2	01010	10	$25(10) - 5(10)^2 = -250$	-250
3	00010	2	$25(2) - 5(2)^2 = 30$	30
4	01111	15	$25(15) - 5(15)^2 = -450$	-450

Genetic Algorithm : Example

Step 8: Repeat

Repeat the process (selection, crossover, mutation) for several generations until convergence.

Optimal Solution

The maximum value of $f(x) = 25x - 5x^2$ occurs at $x = 2.5$. Since x must be an integer, the optimal solution is $x = 2$ or $x = 3$:

- $f(2) = 25(2) - 5(2)^2 = 30$
- $f(3) = 25(3) - 5(3)^2 = 30$

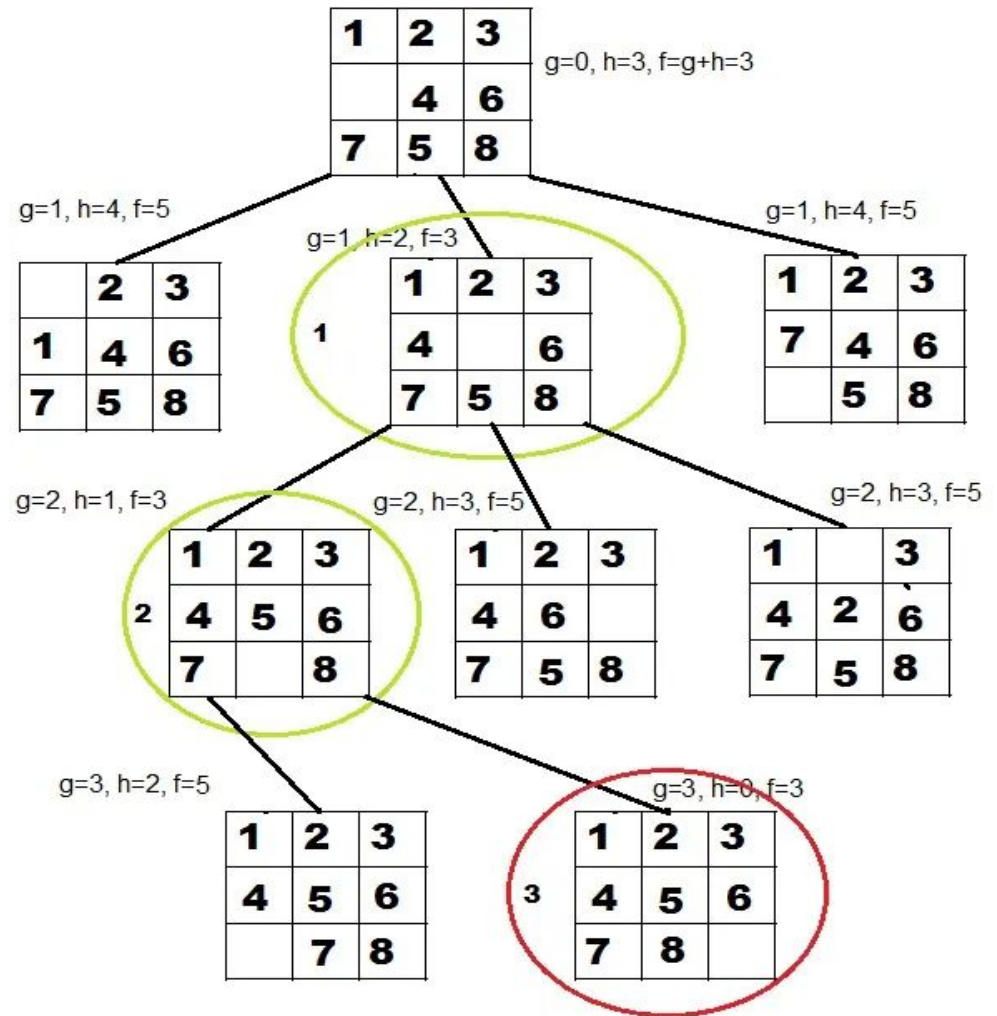
Extra Questions

1. 8 Puzzle with Heuristic (A* Search)

Where,

- h-scores is the number of misplaced tile
- g-score will remain as the number of nodes traversed from a start node to get to the current node.

$$f\text{-score} = h\text{-score} + g\text{-score}$$



Extra Questions

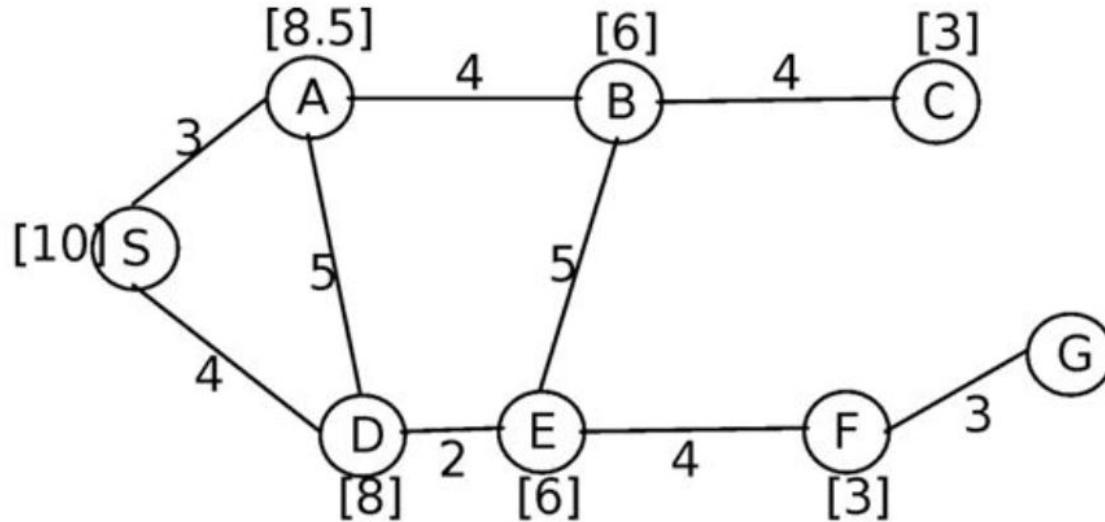
2. Water Jug Problem

The sequence of applied rules:

- Fill the 3G jug from pump
- Empty the 3G jug into 4G jug
- Fill the 3G jug from the pump
- Fill the 4G jug from the 3G jug

Action	Condition (If...)	Resulting State (Then...)
Fill 4G Jug	$x < 4$	$(4, y)$
Fill 3G Jug	$y < 3$	$(x, 3)$
Empty 4G Jug	$x > 0$	$(0, y)$
Empty 3G Jug	$y > 0$	$(x, 0)$
Pour 3G to 4G (Fills 4G completely)	$x + y \geq 4, y > 0$	$(4, y - (4 - x))$
Pour 4G to 3G (Fills 3G completely)	$x + y \geq 3, x > 0$	$(x - (3 - y), 3)$
Pour 3G to 4G (All fits)	$x + y \leq 4, y > 0$	$(x + y, 0)$
Pour 4G to 3G (All fits)	$x + y \leq 3, x > 0$	$(0, x + y)$

Hill-climbing search algorithm: Example



Hill-climbing search algorithm: Example

